

1 Computational Geometry

1.1 basic

```

1 struct point {
2     double x, y;
3     point(double x = 0, double y = 0) : x(x), y(y) {}
4     point operator - (const point& b) const {
5         return point(x - b.x, y - b.y); }
6     point operator + (const point& b) const {
7         return point(x + b.x, y + b.y); }
8     point operator * (double k) const {
9         return point(x * k, y * k); }
10    bool operator < (const point& b) {
11        if (x != b.x) return x < b.x;
12        return y < b.y;
13    }
14    double cross(point a, point b) {return a.x *
15        b.y - a.y * b.x;}
16    // 直線ab與直線cd的交點
17    point intersect(point a, point b, point c,
18        point d)
19    {
20        double c1 = cross(b - a, c - a);
21        double c2 = cross(b - a, d - a);
22        return c + (d - c) * (c1 / (c1 - c2));
23    }
24 }

```

1.2 convex-hull

```

1 double cross(point a, point b) {return a.x *
2     b.y - a.y * b.x;}
3 vector<point> build(vector<point> v)
4 {
5     vector<point> ans;
6     sort(v.begin(), v.end());
7     for(int i = 0; i < v.size(); i++){
8         while(ans.size() > 1 and cross(ans.
9             end()[-2] - ans.end()[-1], ans.
10             end()[-2] - v[i]) > 0) ans.
11             pop_back();
12         ans.pb(v[i]);
13     }
14     for(int i = v.size() - 2; i >= 0; i--){
15         while(ans.size() > 1 and cross(ans.
16             end()[-2] - ans.end()[-1], ans.
17             end()[-2] - v[i]) > 0) ans.
18             pop_back();
19         ans.pb(v[i]);
20     }
21     ans.pop_back();
22     return ans;
23 }

```

1.3 Line Segment Intersection

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4     (0);
5 #define pb push_back
6 #define pii pair<int, int>
7 #define F first
8 #define S second
9 #define int long long
10 pii f(pii a, pii b)
11 {
12     return {b.F - a.F, b.S - a.S};
13 }
14 int across(pii a, pii b)
15 {
16     if(a.F * b.S < a.S * b.F) return -1;
17     if(a.F * b.S > a.S * b.F) return 1;
18     return 0;
19 }
20 bool overlap(pii a, pii b, pii c, pii d)
21 {
22     if(a.F == b.F and b.F == c.F and c.F ==
23         d.F){
24         if(min(a.S, b.S) > max(c.S, d.S) or
25             min(c.S, d.S) > max(a.S, b.S))
26             return false;
27     }
28     if(min(a.F, b.F) > max(c.F, d.F) or min(
29         c.F, d.F) > max(a.F, b.F)) return
30         false;
31     return true;
32 }
33 void solve()
34 {
35     int x1, x2, x3, x4, y1, y2, y3, y4;
36     cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3
37         >> x4 >> y4;
38     pii a = {x1, y1};
39     pii b = {x2, y2};
40     pii c = {x3, y3};
41     pii d = {x4, y4};
42     int x = across(f(a, b), f(a, c));
43     int y = across(f(a, b), f(a, d));
44     int z = across(f(c, d), f(c, a));
45     int w = across(f(c, d), f(c, b));
46     if(x * y < 0 and z * w < 0) cout << "YES
47         \n";
48     else if(x * y <= 0 and z * w <= 0 and
49         overlap(a, b, c, d))cout << "YES\n";
50     else cout << "NO\n";
51 }
52 signed main()
53 {
54     KCC
55     int t;
56     cin >> t;
57     while(t--) solve();
58     return 0;
59 }

```

1.4 Point in Polygon

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4     (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define F first
8 #define S second
9 #define int long long
10 int x[(int)1e3 + 5], y[(int) 1e3 + 5], n, m;
11 bool onsegment(pii a, pii b, pii p)
12 {
13     if((a.F - b.F) * (a.S - p.S) != (a.F - p
14         .F) * (a.S - b.S)) return false;
15     if(!(min(a.F, b.F) <= p.F and p.F <= max
16         (a.F, b.F))) return false;
17     if(!(min(a.S, b.S) <= p.S and p.S <= max
18         (a.S, b.S))) return false;
19     return true;
20 }
21 signed main()
22 {
23     KCC
24     cin >> n >> m;
25     for(int i = 0; i < n; i++) cin >> x[i]
26         >> y[i];
27     while(m--){
28         int a, b;
29         cin >> a >> b;
30         pii p = {a, b};
31         bool boundary = false;
32         int cnt = 0;
33         for(int i = 0; i < n; i++){
34             pii u = {x[i], y[i]}, v = {x[(i
35                 + 1) % n], y[(i + 1) % n]};
36             if(onsegment(u, v, p)){
37                 boundary = true;
38                 break;
39             }
40             if(min(u.S, v.S) <= b and b <
41                 max(u.S, v.S)){
42                 /*
43                 Line u_to_v:
44                 X = u.F + t(v.F - u.F)
45                 Y = u.S + t(v.S - u.S)
46                 b = u.S + t(v.S - u.S)
47                 t = (b - u.S) / (v.S - u.S)
48                 X = u.F + (b - u.S) / (v.S -
49                     u.S) * (v.F - u.F)
50                 */
51                 double X = u.F + 1.0 * (b -
52                     u.S) / (v.S - u.S) * (v.
53                         F - u.F);
54                 if(a <= X) cnt++;
55             }
56         }
57         if (boundary) cout << "BOUNDARY\n";
58         else if (cnt & 1) cout << "INSIDE\n";
59         else cout << "OUTSIDE\n";
60     }
61 }

```

1.5 動態開點李超樹

```

1 struct Line{
2     int k, b;
3     int val(int x){
4         return k * x + b;
5     }
6 };
7 struct Node{
8     Node *l = nullptr;
9     Node *r = nullptr;
10    bool has_line = false;
11    Line line;
12 } *root = nullptr; //維護最大值
13 void insert(Node *&node, int l, int r, Line
14     new_line)
15 {
16     if(!node) node = new Node();
17     if(!node->has_line){
18         node->line = new_line;
19         node->has_line = true;
20         return;
21     }
22     int mid = (l + r) >> 1;
23     bool l_better = new_line.val(l) > node->
24         line.val(l);
25     bool mid_better = new_line.val(mid) >
26         node->line.val(mid);
27     if(mid_better) swap(new_line, node->line);
28     if(l == r) return;
29     if(l_better != mid_better) insert(node->
30         l, l, mid, new_line);
31     else insert(node->r, mid + 1, r,
32         new_line);
33 }
34 int query(Node *node, int l, int r, int x)
35 {
36     if(!node) return -2e18;
37     if(!node->has_line) return -2e18;
38     int re = node->line.val(x);
39     if(l == r) return re;
40     int mid = (l + r) >> 1;
41     if(x <= mid) re = max(re, query(node->l,
42         l, mid, x));
43     else re = max(re, query(node->r, mid +
44         1, r, x));
45     return re;
46 }

```

1.6 半平面交集

```

1 // 用有向線段 AB 切割多邊形 v，保留左半邊
2 vector<point> cut(vector<point> v, point a,
3     point b)
4 {
5     vector<point> ans;
6     int n = v.size();
7     for(int i = 0; i < n; i++){
8         point c = v[i], d = v[(i + 1) % n];
9     }

```

```

10 double side_c = cross(b - a, c - a);
11 double side_d = cross(b - a, d - a);
12 if(side_c >= 0){
13     ans.pb(c);
14     if(side_d < 0) ans.pb(intersect(
15         a, b, c, d));
16 }
17 else{
18     if(side_d >= 0) ans.pb(intersect(
19         a, b, c, d));
20 }
21 }
22 return ans;
23 }

```

1.7 最近點對

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define F first
8 #define S second
9 #define int long long
10 pii arr[(int)2e5 + 5];
11 int dis(int i, pii w)
12 {
13     return (arr[i].F - w.S) * (arr[i].F - w.
14         S) + (arr[i].S - w.F) * (arr[i].S -
15         w.F);
16 }
17 signed main()
18 {
19     KCC
20     int n, ans = 8e18, id = 0;
21     cin >> n;
22     for(int i = 0; i < n; i++) cin >> arr[i]
23         .F >> arr[i].S;
24     sort(arr, arr + n);
25     set<pii> s; // {y, x}
26     for(int i = 0; i < n; i++){
27         int d = sqrt(ans) + 1;
28         while(s.size() > 0 and arr[i].F -
29             arr[id].F >= d){
30             s.erase({arr[id].S, arr[id].F});
31             id++;
32         }
33         auto l = s.lower_bound({arr[i].S - d
34             , -1e9});
35         auto r = s.lower_bound({arr[i].S + d
36             , 1e9});
37         for(auto j = l; j != r; ++j) ans =
38             min(ans, dis(i, *j));
39         s.insert({arr[i].S, arr[i].F});
40     }
41     cout << ans << "\n";
42     return 0;
43 }

```

1.8 李超線段樹

```

1 struct Line{
2     int k, b;
3     int val(int x){
4         return k * x + b;
5     }
6 };
7 Line tree[(int)1e6]; //維護最大值
8 bool has_line[(int)1e6];
9 void insert(int node, int l, int r, Line
10     new_line)
11 {
12     if(!has_line[node]){
13         tree[node] = new_line;
14         has_line[node] = true;
15         return;
16     }
17     int mid = (l + r) >> 1;
18     bool l_better = new_line.val(l) > tree[
19         node].val(l);
20     bool mid_better = new_line.val(mid) >
21         tree[node].val(mid);
22     if(mid_better) swap(new_line, tree[node
23         ]);
24     if(l == r) return;
25     if(l_better != mid_better) insert(node
26         << 1, l, mid, new_line);
27     else insert(node << 1 | 1, mid + 1, r,
28         new_line);
29 }
30 int query(int node, int l, int r, int x)
31 {
32     if(!has_line[node]) return -2e18;
33     int re = tree[node].val(x);
34     if(l == r) return re;
35     int mid = (l + r) >> 1;
36     if(x <= mid) re = max(re, query(node <<
37         1, l, mid, x));
38     else re = max(re, query(node << 1 | 1,
39         mid + 1, r, x));
40     return re;
41 }

```

1.9 矩形覆蓋面積

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define F first
8 #define S second
9 #define int long long
10 struct SEG{
11     int cnt, len;
12 }seg[(int)1e7];
13 void pull(int id, int l, int r)
14 {

```

```

15 if(seg[id].cnt > 0) seg[id].len = r - l
16     + 1;
17 else{
18     if(l != r) seg[id].len = seg[id <<
19         1].len + seg[id << 1 | 1].len;
20     else seg[id].len = 0;
21 }
22 void update(int ql, int qr, int x, int l,
23     int r, int id)
24 {
25     if(ql <= l and r <= qr) {
26         seg[id].cnt += x;
27         pull(id, l, r);
28         return;
29     }
30     int mid = (l + r) >> 1;
31     if(ql <= mid) update(ql, qr, x, l, mid,
32         id << 1);
33     if(qr > mid) update(ql, qr, x, mid + 1,
34         r, id << 1 | 1);
35     pull(id, l, r);
36 }
37 signed main()
38 {
39     KCC
40     int n, ans = 0;
41     cin >> n;
42     vector<array<int, 4>> v;
43     for(int i = 0; i < n; i++){
44         int a, b, c, d;
45         cin >> a >> b >> c >> d;
46         a += 1e6 + 1;
47         b += 1e6 + 1;
48         c += 1e6 + 1;
49         d += 1e6 + 1;
50         v.pb({a, b, d, 1});
51         v.pb({c, b, d, -1});
52     }
53     sort(v.begin(), v.end());
54     int last = v[0][0], maxval = 2e6 + 1;
55     for(auto [x, y1, y2, sign] : v){
56         ans += (x - last) * seg[1].len;
57         update(y1, y2 - 1, sign, 1, maxval,
58             1);
59         last = x;
60     }
61     cout << ans << "\n";
62     return 0;
63 }

```

2 Data Structure

2.1 DSU-rollback

```

1 struct DSU_rollback{
2     int components;
3     vector<int> parent, sz;
4     struct modify{
5         int u, v, pre_sz;
6     };

```

```

7     vector<modify> history;
8     void initial(int n){
9         parent.resize(n + 1);
10        sz.resize(n + 1, 1);
11        components = n;
12        for(int i = 1; i <= n; i++) parent[i]
13            = i;
14    }
15    int find(int x){
16        while(x != parent[x]) x = parent[x];
17        return x;
18    }
19    void unite(int u, int v){
20        int root_u = find(u), root_v = find(
21            v);
22        if(root_u == root_v) return;
23        if(sz[root_u] < sz[root_v]) swap(
24            root_u, root_v);
25        history.pb({root_u, root_v, sz[
26            root_u]});
27        parent[root_v] = root_u;
28        sz[root_u] += sz[root_v];
29        components--;
30    }
31    void rollback(){
32        if(history.size() == 0) return;
33        modify last = history.back();
34        history.pop_back();
35        parent[last.v] = last.v;
36        sz[last.u] = last.pre_sz;
37        components++;
38    }
39    int dsutime(){return (int)history.size()
40        ;}
41    void return_to_t(int t){
42        while(history.size() > t) rollback()
43            ;
44    }
45 };

```

2.2 FHQ-treap

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<int, int>
7 #define F first
8 #define S second
9 mt19937 rnd(time(0));
10 const int MAXN = 2e5 + 5;
11 int left_node[MAXN], right_node[MAXN],
12     priority[MAXN], sz[MAXN], node_cnt = 0,
13     root = 0;
14 char val[MAXN];
15 void up(int i)
16 {
17     sz[i] = sz[left_node[i]] + sz[right_node
18         [i]] + 1;
19 }
20 void split(int i, int l, int r, int rank)
21 {

```

2.3 persistent-segment-tree

```

18 if(i == 0){
19     left_node[r] = right_node[l] = 0;
20     return;
21 }
22 if(sz[left_node[i]] + 1 <= rank){
23     right_node[l] = i;
24     split(right_node[i], i, r, rank - sz
25         [left_node[i]] - 1);
26 }
27 else{
28     left_node[r] = i;
29     split(left_node[i], l, i, rank);
30 }
31 up(i);
32 int merge(int l, int r)
33 {
34     if(l == 0 or r == 0) return l + r;
35     if(priority[l] >= priority[r]){
36         right_node[l] = merge(right_node[l],
37             r);
38         up(l);
39         return l;
40     }
41     else{
42         left_node[r] = merge(l, left_node[r]
43             );
44         up(r);
45         return r;
46     }
47 }
48 void out(int i)
49 {
50     if(i == 0) return;
51     out(left_node[i]);
52     cout << val[i];
53     out(right_node[i]);
54 }
55 int main()
56 {
57     KCC
58     int n, m;
59     cin >> n >> m;
60     for(int i = 0; i < n; i++){
61         char c;
62         cin >> c;
63         priority[++node_cnt] = rnd();
64         val[node_cnt] = c;
65         sz[node_cnt] = 1;
66         root = merge(root, node_cnt);
67     }
68     while(m--){
69         int a, b;
70         cin >> a >> b;
71         split(root, 0, 0, b);
72         int lm = right_node[0];
73         int r = left_node[0];
74         split(lm, 0, 0, a - 1);
75         int l = right_node[0];
76         int m = left_node[0];
77         root = merge(merge(l, r), m);
78     }
79     out(root);
80     return 0;
81 }

```

```

1 int upd(int pre, int l, int r, int pos, int
2     val)
3 {
4     int node = ++cnt_tree;
5     tree[node] = tree[pre];
6     if(l == r){
7         tree[node].sum += val;
8         return node;
9     }
10    int mid = (l + r) >> 1;
11    if(pos <= mid) tree[node].l = upd(tree[
12        pre].l, l, mid, pos, val);
13    else tree[node].r = upd(tree[pre].r, mid
14        + 1, r, pos, val);
15    tree[node].sum = tree[tree[node].l].sum
16        + tree[tree[node].r].sum;
17    return node;
18 }
19 int query(int node, int l, int r, int ql,
20     int qr)
21 {
22     if(ql <= l and r <= qr) return tree[node
23         ].sum;
24     int mid = (l + r) >> 1, re = 0;
25     if(ql <= mid) re += query(tree[node].l,
26         l, mid, ql, qr);
27     if(qr > mid) re += query(tree[node].r,
28         mid + 1, r, ql, qr);
29     return re;
30 }
31 int get_kth(int node_l, int node_r, int l,
32     int r, int k)
33 {
34     //原陣列位置區間 (node_l, node_r] 第 k 小
35     /*
36     if(l == r) return l;
37     int mid = (l + r) >> 1;
38     int left_cnt = tree[tree[node_r].l].sum
39         - tree[tree[node_l].l].sum;
40     if(k <= left_cnt) return get_kth(tree[
41         node_l].l, tree[node_r].l, l, mid, k
42         );
43     else return get_kth(tree[node_l].r, tree
44         [node_r].r, mid + 1, r, k - left_cnt
45         );
46     */
47     while (l < r) {
48         int mid = (l + r) >> 1;
49         int left_cnt = tree[tree[node_r].l].
50             sum - tree[tree[node_l].l].sum;
51         if (k <= left_cnt) {
52             node_l = tree[node_l].l;
53             node_r = tree[node_r].l;
54             r = mid;
55         }
56         else {
57             k -= left_cnt;
58             node_l = tree[node_l].r;
59             node_r = tree[node_r].r;
60             l = mid + 1;
61         }
62     }
63     return l;
64 }

```

2.4 segment tree lazytag

```

1 struct SegmentTree{
2     struct Node{
3         int sum, f, d, len;
4     } tree[MAXN << 2];
5     void pull(int id){
6         tree[id].sum = tree[id << 1].sum +
7             tree[id << 1 | 1].sum;
8     }
9     void addtag(int id, int a, int d){
10        tree[id].f += a;
11        tree[id].d += d;
12        tree[id].sum += (2 * a + (tree[id].
13            len - 1) * d) * tree[id].len /
14            2;
15    }
16    void push(int id){
17        if(!tree[id].f && !tree[id].d)
18            return;
19        int mid_f = tree[id].f + tree[id <<
20            1].len * tree[id].d;
21        addtag(id << 1, tree[id].f, tree[id
22            ].d);
23        addtag(id << 1 | 1, mid_f, tree[id].
24            d);
25        tree[id].f = tree[id].d = 0;
26    }
27    void build(int l, int r, int id, int ar
28        []){
29        tree[id].len = r - l + 1;
30        tree[id].f = tree[id].d = 0;
31        if(l == r){
32            tree[id].sum = ar[l];
33            return;
34        }
35        int mid = (l + r) >> 1;
36        build(l, mid, id << 1, ar);
37        build(mid + 1, r, id << 1 | 1, ar);
38        pull(id);
39    }
40    void update(int ql, int qr, int a, int d
41        , int l, int r, int id){
42        if(ql <= l and r <= qr){
43            addtag(id, a + (l - ql) * d, d);
44            return;
45        }
46        push(id);
47        int mid = (l + r) >> 1;
48        if(ql <= mid) update(ql, qr, a, d, l
49            , mid, id << 1);
50        if(qr > mid) update(ql, qr, a, d,
51            mid + 1, r, id << 1 | 1);
52        pull(id);
53    }
54    int query(int ql, int qr, int l, int r,
55        int id){
56        if(ql <= l and r <= qr) return tree[
57            id].sum;
58        push(id);
59        int mid = (l + r) >> 1, re = 0;
60        if(ql <= mid) re += query(ql, qr, l,
61            mid, id << 1);
62        if(qr > mid) re += query(ql, qr, mid
63            + 1, r, id << 1 | 1);
64    }
65 }

```

```

49     return re;
50 }
51 };

```

2.5 trie

```

1 struct Node{
2     int next_node[2]; // 01-trie
3     int max_idx;
4     Node() {
5         next_node[0] = next_node[1] = -1;
6         max_idx = -1;
7     }
8 };
9 vector<Node> trie;
10 void init_trie()
11 {
12     trie.clear();
13     trie.emplace_back(); // 插入根節點
14 }
15 void insert(int x, int id)
16 {
17     int u = 0;
18     for(int i = 29; i >= 0; i--){
19         int bit = (x >> i) & 1;
20         if(trie[u].next_node[bit] == -1){
21             trie[u].next_node[bit] = trie.
22                 size();
23             trie.emplace_back();
24         }
25         u = trie[u].next_node[bit];
26         trie[u].max_idx = max(trie[u].
27             max_idx, id);
28     }
29 }
30 // 減少空間的存法
31 struct Edge {
32     int to, nxt;
33     char c;
34 };
35 struct Node {
36     int head = -1;
37     // 放節點要存的資訊
38 };
39 vector<Node> trie;
40 vector<Edge> edges;
41 int get_child(int u, char c){
42     for(int e = trie[u].head; e != -1; e =
43         edges[e].nxt){
44         if(edges[e].c == c) return edges[e].
45             to;
46     }
47     return -1;
48 }
49 int nxt_child(int u, char c){
50     int v = get_child(u, c);
51     if(v != -1) return v;
52     v = trie.size();
53     trie.emplace_back();
54 }

```

```

52 |
53 | edges.push_back({v, trie[u].head, c});
54 | trie[u].head = (int)edges.size() - 1;
55 | return v;
56 | }

```

2.6 Weighted DSU

```

1 | struct Weighted_DSU{
2 |     vector<int> parent, dis;
3 |     void initial(int n){
4 |         parent.resize(n + 1);
5 |         dis.resize(n + 1, 0);
6 |         for(int i = 1; i <= n; i++) parent[i] = i;
7 |     }
8 |     int find(int x){
9 |         if(parent[x] != x){
10 |             int tmp = parent[x];
11 |             parent[x] = find(tmp);
12 |             dis[x] += dis[tmp];
13 |         }
14 |         return parent[x];
15 |     }
16 |     void unite(int l, int r, int d){
17 |         int lf = find(l), rf = find(r);
18 |         if(lf != rf){
19 |             parent[lf] = rf;
20 |             dis[lf] = dis[r] - dis[l] + d;
21 |         }
22 |     }
23 |     int query(int l, int r){
24 |         if(find(l) != find(r)) return 1e18;
25 |         return dis[l] - dis[r];
26 |     }
27 | };

```

2.7 動態開點線段樹

```

1 | struct Node{
2 |     int sum = 0;
3 |     Node *l = nullptr;
4 |     Node *r = nullptr;
5 | }*root = nullptr;
6 |
7 | void add(Node *&node, int L, int R, int pos, int val)
8 | {
9 |     if(!node) node = new Node();
10 |    node->sum += val;
11 |    if(L == R) return;
12 |    int mid = (L + R) / 2;
13 |    if(pos <= mid) add(node->l, L, mid, pos, val);
14 |    else add(node->r, mid + 1, R, pos, val);
15 | }
16 | int query(Node *node, int L, int R, int ql, int qr)
17 | {
18 |     if(!node) return 0;

```

```

19 |     if(ql <= L and R <= qr) return node->sum;
20 |     ;
21 |     int mid = (L + R) / 2, ans = 0;
22 |     if(ql <= mid) ans += query(node->l, L, mid, ql, qr);
23 |     if(qr > mid) ans += query(node->r, mid + 1, R, ql, qr);
24 |     return ans;

```

3 Flow

3.1 dinic

```

1 | struct Dinic{
2 |     struct Edge{
3 |         int to, cap, rev;
4 |     };
5 |     vector<vector<Edge>> adj;
6 |     vector<int> level, ptr;
7 |     int n;
8 |     Dinic(int _n){
9 |         n = _n;
10 |        level.resize(n + 1);
11 |        ptr.resize(n + 1);
12 |        adj.resize(n + 1);
13 |    }
14 |    void addedge(int u, int v, int w){
15 |        adj[u].pb({v, w, adj[v].size()});
16 |        adj[v].pb({u, 0, adj[u].size() - 1});
17 |    }
18 |    bool bfs(int s, int t)
19 |    {
20 |        fill(level.begin(), level.end(), -1);
21 |        ;
22 |        level[s] = 0;
23 |        queue<int> q;
24 |        q.push(s);
25 |        while(q.size()){
26 |            int x = q.front(); q.pop();
27 |            for(auto edge : adj[x]){
28 |                if(edge.cap > 0 and level[edge.to] == -1){
29 |                    level[edge.to] = level[x] + 1;
30 |                    q.push(edge.to);
31 |                }
32 |            }
33 |        }
34 |        return (level[t] != -1);
35 |    }
36 |    int dfs(int now, int t, int pushed)
37 |    {
38 |        if(now == t) return pushed;
39 |        for(int &i = ptr[now]; i < adj[now].size(); i++){
40 |            auto &w = adj[now][i];
41 |            if(level[now] + 1 != level[w.to] or w.cap == 0) continue;

```

```

42 |            int tr = dfs(w.to, t, min(pushed, w.cap));
43 |            if(tr == 0) continue;
44 |            w.cap -= tr;
45 |            adj[w.to][w.rev].cap += tr;
46 |            return tr;
47 |        }
48 |        return 0;
49 |    }
50 |    int compute_max_flow(){
51 |        int max_flow = 0;
52 |        while(bfs(1, n)){
53 |            fill(ptr.begin(), ptr.end(), 0);
54 |            while(int pushed = dfs(1, n, 1e18)){
55 |                max_flow += pushed;
56 |            }
57 |        }
58 |        return max_flow;
59 |    }

```

3.2 dinic flow decomposition

```

1 | struct dinic{
2 |     struct edge{
3 |         int to, cap, rev;
4 |     };
5 |     vector<vector<edge>> adj;
6 |     vector<int> level, ptr;
7 |     int n;
8 |     vector<vector<int>> fwd;
9 |     dinic(int _n){
10 |         n = _n;
11 |         level.resize(n+1);
12 |         ptr.resize(n+1);
13 |         adj.resize(n+1);
14 |         fwd.resize(n+1);
15 |     }
16 |     void addedge(int u, int v, int w){
17 |         fwd[u].push_back({adj[u].size()});
18 |         adj[u].push_back({v, w, adj[v].size()});
19 |         adj[v].push_back({u, 0, adj[u].size() - 1});
20 |     }
21 |     bool bfs(int s, int t){
22 |         fill(level.begin(), level.end(), -1);
23 |         ;
24 |         level[s] = 0;
25 |         queue<int> q;
26 |         q.push(s);
27 |         while(q.size()){
28 |             int x = q.front(); q.pop();
29 |             for(auto edge : adj[x]){
30 |                 if(edge.cap > 0 and level[edge.to] == -1){
31 |                     level[edge.to] = level[x] + 1;
32 |                     q.push(edge.to);
33 |                 }
34 |             }

```

```

35 |         return (level[t] != -1);
36 |     }
37 |     int dfs(int now, int t, int pushed){
38 |         if(now == t) return pushed;
39 |         for(int &i = ptr[now]; i < adj[now].size(); i++){
40 |             auto &w = adj[now][i];
41 |             if(level[now] + 1 != level[w.to] or w.cap == 0) continue;
42 |             int tr = dfs(w.to, t, min(pushed, w.cap));
43 |             if(tr == 0) continue;
44 |             w.cap -= tr;
45 |             adj[w.to][w.rev].cap += tr;
46 |             return tr;
47 |         }
48 |         return 0;
49 |     }
50 |     int compute(){
51 |         int max_flow = 0;
52 |         while(bfs(1, n)){
53 |             fill(ptr.begin(), ptr.end(), 0);
54 |             while(int pushed = dfs(1, n, 1e18)){
55 |                 max_flow += pushed;
56 |             }
57 |         }
58 |         return max_flow;
59 |     }
60 | }
61 | int &flowof(int u, int idx){
62 |     auto &e = adj[u][idx];
63 |     return adj[e.to][e.rev].cap;
64 | }
65 | // flow, path
66 | vector<pair<int, vector<int>>> decompose(int s, int t){
67 |     vector<pair<int, vector<int>>> paths;
68 |     ;
69 |     vector<int> cur(n+1, 0), pos(n+1);
70 |     while(true){
71 |         vector<int> path = {s};
72 |         vector<pair<int, int>> used; //u, idx
73 |         fill(pos.begin(), pos.end(), -1);
74 |         ;
75 |         pos[s] = 0;
76 |         int u = s;
77 |         bool cycle = false;
78 |         while(u != t){
79 |             while(cur[u] < fwd[u].size() and flowof(u, fwd[u][cur[u]]) == 0) cur[u]++;
80 |             if(cur[u] == fwd[u].size()) break;
81 |             int idx = fwd[u][cur[u]];
82 |             int v = adj[u][idx].to;
83 |             used.push_back({u, idx});
84 |             if(pos[v] != -1){ cycle = true; u = v; break; }
85 |             pos[v] = path.size();
86 |             path.push_back(v);
87 |             u = v;
88 |         }
89 |         if(used.empty()) break;
90 |         int start = cycle ? pos[u] : 0;

```

```

89     int x = LLONG_MAX;
90     for(int i = start; i < used.size(); i++) x = min(x, flowof(used
91         [i].first, used[i].second));
92     for(int i = start; i < used.size(); i++) flowof(used[i].first,
93         used[i].second) -= x;
94     if(!cycle) paths.push_back({x,
95         path});
96     return paths;
};

```

3.3 Edmonds-Karp

```

1 struct max_flow{
2     int n;
3     vector<int> dis, par, mn;
4     vector<vector<int>> g;
5     vector<array<int, 3>> edge;
6     max_flow(int _n){
7         n = _n;
8         dis.resize(n + 5);
9         par.resize(n + 5);
10        mn.resize(n + 5);
11        g.resize(n + 5);
12    }
13    void add_edge(int a, int b, int c){
14        g[a].pb(edge.size());
15        edge.pb({a, b, c});
16        g[b].pb(edge.size());
17        edge.pb({b, a, 0});
18    }
19    int bfs()
20    {
21        for(int i = 1; i <= n; i++) dis[i] =
22            -1;
23        for(int i = 1; i <= n; i++) mn[i] =
24            1e18;
25        dis[1] = 0;
26        queue<int> q;
27        q.push(1);
28        while(q.size()){
29            int x = q.front(); q.pop();
30            for(int i : g[x]){
31                auto [now, to, w] = edge[i];
32                if(dis[to] == -1 and w > 0){
33                    dis[to] = dis[now] + 1;
34                    par[to] = i;
35                    mn[to] = min(mn[now], w);
36                    q.push(to);
37                }
38            }
39        }
40        if(dis[n] == -1) return 0;
41        int now = n;
42        while(now != 1){
43            auto &[a, b, w] = edge[par[now]];
44            auto &[ra, rb, rw] = edge[par[
45                now ^ 1]];

```

```

43        w -= mn[n];
44        rw += mn[n];
45        now = a;
46    }
47    return mn[n];
48    int max_flow_ans(){
49        int ans = 0;
50        while(true){
51            int t = bfs();
52            if(t == 0) break;
53            ans += t;
54        }
55        return ans;
56    }
57    };
58    };

```

3.4 Min Cost Max-Flow

```

1 struct Edge {
2     ll to, cap, cost, rev;
3     Edge() {}
4     Edge(int _to, ll _cap, ll _cost, int
5         _rev) : to(_to), cap(_cap), cost(
6         _cost), rev(_rev) {}
7 };
8 vector<Edge> adj[N];
9 ll dis[N]; int par[N], par_id[N]; bool
10 in_que[N];
11 pair<ll, ll> MCMF(int s, int t) {
12     ll flow = 0, cost = 0;
13     while(true) {
14         memset(dis, 0x3f, sizeof(dis));
15         memset(in_que, 0, sizeof(in_que));
16         queue<int> que;
17         que.push(s); dis[s] = 0;
18         while(!que.empty()) { // SPFA
19             int x = que.front();
20             que.pop(); in_que[x] = 0;
21             for(int i = 0; i < (int)adj[x].
22                 size(); i++) {
23                 Edge &e = adj[x][i];
24                 if(e.cap > 0 && dis[e.to] >
25                     dis[x] + e.cost) {
26                     dis[e.to] = dis[x] + e.
27                         cost;
28                     par[e.to] = x;
29                     par_id[e.to] = i;
30                     if(!in_que[e.to]) {
31                         in_que[e.to] = 1;
32                         que.push(e.to);
33                     }
34                 }
35             }
36         }
37         if(dis[t] >= INF) break;
38         ll mi_flow = INF;
39         for(int i = t; i != s; i = par[i])
40             mi_flow = min(mi_flow, adj[par[i]
41                 ][par_id[i]].cap);
42         flow += mi_flow, cost += mi_flow *
43             dis[t];
44         for(int i = t; i != s; i = par[i]) {

```

```

37         Edge &e = adj[par[i]][par_id[i]
38             []];
39         e.cap -= mi_flow;
40         adj[e.to][e.rev].cap += mi_flow;
41     }
42     return make_pair(flow, cost);
43 }
44 void add_edge(int a, int b, ll cap, ll cost)
45 {
46     adj[a].push_back(Edge(b, cap, cost, (int)
47         adj[b].size()));
48     adj[b].push_back(Edge(a, 0, -cost, (int)
49         adj[a].size() - 1));
50 }
51 ll hungarian(vector<vector<ll>> cost) {
52     int n = (int)cost.size();
53     for(int i = 0; i < n; i++) {
54         for(int j = 0; j < n; j++) {
55             int L = i, T = j + n;
56             add_edge(L, T, 1, cost[i][j]);
57         }
58     }
59     int s = 2 * n, t = 2 * n + 1;
60     for(int i = 0; i < n; i++) {
61         int L = i, T = i + n;
62         add_edge(s, L, 1, 0);
63         add_edge(T, t, 1, 0);
64     }
65     // MCMF(s, t)
66     for(int i = 0; i < n; i++) {
67         for(Edge &e : adj[i]) {
68             if(e.to == s) continue;
69             int row = i, col = e.to - n;
70             if(e.cap == 0)
71                 // (row + 1) match (col + 1)
72             }
73         }
74     }
75     return MCMF(s, t).second;
76 }

```

4 Graph

4.1 bellman-ford

```

1 void bellman_ford(){
2     for(int i = 1; i <= n; i++){
3         bool update = false;
4         for(int j = 1; j <= n; j++){
5             for(auto k : g[j])
6                 if(dis[k.F] > dis[j] + k.S)
7                     update = true, dis[k.F]
8                         = dis[j] + k.S;
9         }
10        if(!update)break;
11        if(i == n)cout << "neg_cycle" << "\n";
12    }
13 }

```

4.2 dijk

```

1 priority_queue<pii, vector<pii>, greater<pii>
2     >> q;
3 q.push({0, 1});
4 dis[1] = 0;
5 while(q.size()){
6     auto [d, x] = q.top(); q.pop();
7     if(d > dis[x]) continue;
8     for(auto [to, w] : g[x]){
9         if(dis[to] > dis[x] + w){
10             dis[to] = dis[x] + w;
11             q.push({dis[to], to});
12         }
13     }
14 }

```

4.3 Euler-Path

```

1 void dfs(int x){
2     while(g[x].size()){
3         int nxt = g[x].back();
4         g[x].pop_back();
5         dfs(nxt);
6     }
7     ans.pb(x); // 倒序記錄答案
8 }

```

4.4 SCC

```

1 struct SCC{
2     vector<vector<int>> g, rg;
3     vector<bool> vis, used;
4     vector<int> component, st;
5     int component_cnt = 0, n;
6     void dfs1(int node){
7         vis[node] = true;
8         for(int i : g[node]) if(!vis[i])
9             dfs1(i);
10        st.pb(node);
11    }
12    void dfs2(int node){
13        used[node] = true;
14        for(int i : rg[node]) if(!used[i])
15            dfs2(i);
16        component[node] = component_cnt;
17    }
18    SCC(const vector<vector<int>> v){
19        n = v.size() - 1;
20        g = v;
21        rg.resize(n + 1);
22        vis.resize(n + 1, false);
23        used.resize(n + 1, false);
24        component.resize(n + 1, 0);
25        for(int i = 1; i <= n; i++) for(int
26            j : v[i]) rg[j].pb(i);
27    }
28    vector<int> find_component(){

```

```

26 for(int i = 1; i <= n; i++) if(!vis[
    i]) dfs1(i);
27 reverse(st.begin(), st.end());
28 for(int i : st) if(!used[i]){
29     component_cnt++;
30     dfs2(i);
31 }
32 return component;
33 }
34 };

```

4.5 tarjan scc

```

1 struct tarjan_scc{
2     int n, dfn_cnt = 0, scc_cnt = 0;
3     vector<int> belong, dfn, low, stack;
4     vector<vector<int>> g;
5     void initial(int _n){
6         n = _n;
7         belong.resize(n + 1, 0);
8         dfn.resize(n + 1, 0);
9         low.resize(n + 1, 0);
10        g.resize(n + 1, vector<int>(0));
11    }
12    void addedge(int u, int v){
13        g[u].pb(v);
14    }
15    void tarjan(int u){
16        dfn[u] = low[u] = ++dfn_cnt;
17        stack.pb(u);
18        for(int i : g[u]){
19            if(dfn[i] == 0){
20                tarjan(i);
21                low[u] = min(low[u], low[i]);
22            }
23            else if(belong[i] == 0){
24                low[u] = min(low[u], dfn[i]);
25            }
26        }
27        if(dfn[u] == low[u]){
28            scc_cnt++;
29            int x;
30            do{
31                x = stack.back();
32                belong[x] = scc_cnt;
33                stack.pop_back();
34            }while(x != u);
35        }
36    }
37    void work(){
38        for(int i = 1; i <= n; i++){
39            if(belong[i] == 0){
40                tarjan(i);
41            }
42        }
43    }
44 };

```

4.6 線段樹優化建圖

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define int long long
8 #define vvi vector<vector<long long>>
9 #define ll long long
10 const int INF = 2e18;
11 vector<pii> g[(int)5e5 + 5];
12
13 int n, m;
14 void add(int u, int v, int w)
15 {
16     g[u].pb(pii{v, w});
17 }
18 void build(int l, int r, int id)
19 {
20     if(l == r){
21         add(l, n + id, 0);
22         add(n + id, l, 0);
23         return;
24     }
25     int mid = (l + r) >> 1;
26     build(l, mid, id << 1);
27     build(mid + 1, r, id << 1 | 1);
28     add(n + id, n + (id << 1), 0);
29     add(n + id, n + (id << 1 | 1), 0);
30 }
31 // 建立 u -> [ql, qr] 邊權w的有向邊
32 void update(int l, int r, int id, int ql,
33             int qr, int u, int w)
34 {
35     if(ql <= l and r <= qr){
36         add(u, n + id, w);
37         return;
38     }
39     int mid = (l + r) >> 1;
40     if(ql <= mid) update(l, mid, id << 1, ql, qr, u, w);
41     if(qr > mid) update(mid + 1, r, id << 1 | 1, ql, qr, u, w);
42 }
43 int dis[(int)5e5 + 5];
44 void Dijk()
45 {
46     priority_queue<pii, vector<pii>, greater
47     <pii>> q;
48     q.push({0, 1});
49     dis[1] = 0;
50     while(q.size()){
51         auto [d, x] = q.top(); q.pop();
52         if(d > dis[x]) continue;
53         for(auto [to, w] : g[x]){
54             if(dis[to] > dis[x] + w){
55                 dis[to] = dis[x] + w;
56                 q.push({dis[to], to});
57             }
58         }
59     }
60 }

```

```

59 signed main()
60 {
61     KCC
62     cin >> n >> m;
63     build(1, n, 1);
64     for(int i = 0; i < m; i++){
65         int op; cin >> op;
66         if(op == 1){
67             int u, v, w;
68             cin >> u >> v >> w;
69             g[u].pb(pii{v, w});
70         }
71         else{
72             int u, l, r, w;
73             cin >> u >> l >> r >> w;
74             update(1, n, 1, l, r, u, w);
75         }
76     }
77     for(int i = 1; i <= 5 * n; i++) dis[i] =
78         1e18;
79     Dijk();
80     for(int i = 1; i <= n; i++) cout << (dis
81         [i] == 1e18 ? -1 : dis[i]) << " ";
82 }

```

5 Linear Programming

5.1 simplex

```

1 /*target:
2 max \sum_{j=1}^n A_{0,j}*x_j
3 condition:
4 \sum_{j=1}^n A_{i,j}*x_j <= A_{i,0} | i=1~m
5 x_j >= 0 | j=1~n
6 VDB = vector<double>*/
7 template<class VDB>
8 VDB simplex(int m,int n,vector<VDB> a){
9     vector<int> left(m+1), up(n+1);
10    iota(left.begin(), left.end(), n);
11    iota(up.begin(), up.end(), 0);
12    auto pivot = [&](int x, int y){
13        swap(left[x], up[y]);
14        auto k = a[x][y]; a[x][y] = 1;
15        vector<int> pos;
16        for(int j = 0; j <= n; ++j){
17            a[x][j] /= k;
18            if(a[x][j] != 0) pos.push_back(j);
19        }
20        for(int i = 0; i <= m; ++i){
21            if(a[i][y]==0 || i == x) continue;
22            k = a[i][y], a[i][y] = 0;
23            for(int j : pos) a[i][j] -= k*a[x][j];
24        }
25    };
26    for(int x,y;;){
27        for(int i=x+1; i <= m; ++i)
28            if(a[i][0]<a[x][0]) x = i;
29        if(a[x][0]>=0) break;
30        for(int j=y+1; j <= n; ++j)
31            if(a[x][j]<a[x][y]) y = j;

```

```

32        if(a[x][y]>=0) return VDB();//infeasible
33        pivot(x, y);
34    }
35    for(int x,y;;){
36        for(int j=y+1; j <= n; ++j)
37            if(a[0][j] > a[0][y]) y = j;
38        if(a[0][y]<=0) break;
39        x = -1;
40        for(int i=1; i<=m; ++i) if(a[i][y] > 0)
41            if(x == -1 || a[i][0]/a[i][y]
42                < a[x][0]/a[x][y]) x = i;
43        if(x == -1) return VDB();//unbounded
44        pivot(x, y);
45    }
46    VDB ans(n + 1);
47    for(int i = 1; i <= m; ++i)
48        if(left[i] <= n) ans[left[i]] = a[i][0];
49    ans[0] = -a[0][0];
50    return ans;
51 }

```

6 Number Theory

6.1 basic

```

1 template<typename T>
2 void gcd(const T &a,const T &b,T &d,T &x,T &
3     y){
4     if(!b) d=a,x=1,y=0;
5     else gcd(b,a%b,d,y,x), y-=x*(a/b);
6 }
7 long long int phi[N+1];
8 void phiTable(){
9     for(int i=1;i<=N;i++)phi[i]=i;
10    for(int i=1;i<=N;i++)for(x=i*2;x<=N;x+=i)
11        phi[x]-=phi[i];
12 }
13 void all_divdown(const LL &n) { // all n/x
14     for(LL a=1;a<=n;a=n/(n/(a+1))) {
15         // dosomething;
16     }
17 }
18 const int MAXPRIME = 1000000;
19 int iscom[MAXPRIME], prime[MAXPRIME],
20     primecnt;
21 int phi[MAXPRIME], mu[MAXPRIME];
22 void sieve(void){
23     memset(iscom,0,sizeof(iscom));
24     primecnt = 0;
25     phi[1] = mu[1] = 1;
26     for(int i=2;i<MAXPRIME;++i) {
27         if(!iscom[i]) {
28             prime[primecnt++] = i;
29             mu[i] = -1;
30             phi[i] = i-1;
31         }
32         for(int j=0;j<primecnt;++j) {
33             int k = i * prime[j];
34             if(k>MAXPRIME) break;
35             iscom[k] = prime[j];
36             if(i%prime[j]==0) {

```

```

34     mu[k] = 0;
35     phi[k] = phi[i] * prime[j];
36     break;
37 } else {
38     mu[k] = -mu[i];
39     phi[k] = phi[i] * (prime[j]-1);
40 }
41 }
42 }
43 }
44
45 bool g_test(const LL &g, const LL &p, const
vector<LL> &v) {
46     for(int i=0; i<v.size(); ++i)
47         if(modexp(g, (p-1)/v[i], p) == 1)
48             return false;
49     return true;
50 }
51 LL primitive_root(const LL &p) {
52     if(p==2) return 1;
53     vector<LL> v;
54     Factor(p-1, v);
55     v.erase(unique(v.begin(), v.end()), v.end());
56     for(LL g=2; g<p; ++g)
57         if(g_test(g, p, v))
58             return g;
59     puts("primitive_root NOT FOUND");
60     return -1;
61 }
62 int Legendre(const LL &a, const LL &p) {
63     return modexp(a%p, (p-1)/2, p);
64 }
65 LL inv(const LL &a, const LL &n) {
66     LL d, x, y;
67     gcd(a, n, d, x, y);
68     return d==1 ? (x+n)%n : -1;
69 }
70 int inv[maxN];
71 LL invtable(int n, LL P){
72     inv[1]=1;
73     for(int i=2; i<n; ++i)
74         inv[i]=(P-(P/i))*inv[P%i]%P;
75 }
76
77 LL log_mod(const LL &a, const LL &b, const
LL &p) {
78     // a ^ x = b ( mod p )
79     int m=sqrt(p+.5), e=1;
80     LL v=inv(modexp(a, m, p), p);
81     map<LL, int> x;
82     x[1]=0;
83     for(int i=1; i<m; ++i) {
84         e = LLMul(e, a, p);
85         if(!x.count(e)) x[e] = i;
86     }
87     for(int i=0; i<m; ++i) {
88         if(x.count(b)) return i*m + x[b];
89         b = LLMul(b, v, p);
90     }
91     return -1;
92 }
93
94 LL Tonelli_Shanks(const LL &n, const LL &p)
{

```

```

95 // x^2 = n ( mod p )
96 if(n==0) return 0;
97 if(Legendre(n, p) != 1) while(1) { puts("SQRT
ROOT does not exist"); }
98 int S = 0;
99 LL Q = p-1;
100 while( !(Q&1) ) { Q>>=1; ++S; }
101 if(S==1) return modexp(n%p, (p+1)/4, p);
102 LL z = 2;
103 for(; Legendre(z, p) != -1; ++z)
104     LL c = modexp(z, Q, p);
105 LL R = modexp(n%p, (Q+1)/2, p), t = modexp(n
%p, Q, p);
106 int M = S;
107 while(1) {
108     if(t==1) return R;
109     LL b = modexp(c, 1L<<(M-i-1), p);
110     R = LLMul(R, b, p);
111     t = LLMul(LLmul(b, b, p), t, p);
112     c = LLMul(b, b, p);
113     M = i;
114 }
115 return -1;
116 }
117
118 template<typename T>
119 T Euler(T n){
120     T ans=n;
121     for(T i=2; i*i<=n; ++i){
122         if(n%i==0){
123             ans=ans/i*(i-1);
124             while(n%i==0)n/=i;
125         }
126     }
127     if(n>1)ans=ans/n*(n-1);
128     return ans;
129 }
130
131 //Chinese_remainder_theorem
132 template<typename T>
133 T pow_mod(T n, T k, T m){
134     T ans=1;
135     for(n=(n>m?n%m:n); k>=1){
136         if(k&1)ans=ans*n%m;
137         n=n*n%m;
138     }
139     return ans;
140 }
141
142 template<typename T>
143 T crt(vector<T> &m, vector<T> &a){
144     T M=1, tM, ans=0;
145     for(int i=0; i<(int)m.size(); ++i)M*=m[i];
146     for(int i=0; i<(int)a.size(); ++i){
147         tM=M/m[i];
148         ans=(ans+(a[i]*tM*M)%pow_mod(tM, Euler(m[
i])-1, m[i])%M)%M;
149     }
150     //如果m[i]是質數 · Euler(m[i])-1=m[i]-2 ·
就都不用算Euler了*/
151     return ans;
152 }
153
154 //java code
155 //求sqrt(N)的連分數
156 public static void Pell(int n){

```

```

157 BigInteger N, p1, p2, q1, q2, a0, a1, a2, g1, g2, h1
, h2, p, q;
158 g1=q2=p1=BigInteger.ZERO;
159 h1=q1=p2=BigInteger.ONE;
160 a0=a1=BigInteger.valueOf((int)Math.sqrt
(1.0*n));
161 BigInteger ans=a0.multiply(a0);
162 if(ans.equals(BigInteger.valueOf(n))){
163     System.out.println("No solution!");
164     return ;
165 }
166 while(true){
167     g2=a1.multiply(h1).subtract(g1);
168     h2=N.subtract(g2.pow(2)).divide(h1);
169     a2=g2.add(a0).divide(h2);
170     p=a1.multiply(p2).add(p1);
171     q=a1.multiply(q2).add(q1);
172     if(p.pow(2).subtract(N.multiply(q.pow
(2))).compareTo(BigInteger.ONE)==0)
173         break;
174     g1=g2; h1=h2; a1=a2;
175     p1=p2; p2=p;
176     q1=q2; q2=q;
177 }
178 System.out.println(p+" "+q);

```

6.2 bit set

```

1 void sub_set(int S){
2     int sub=S;
3     do{
4         //對某集合的子集合的處理
5         sub=(sub-1)&S;
6     }while(sub!=S);
7 }
8 void k_sub_set(int k, int n){
9     int comb=(1<<k)-1, S=1<<n;
10    while(comb<S){
11        //對大小為k的子集合的處理
12        int x=comb&-comb, y=comb+x;
13        comb=((comb&y)/x>>1)|y;
14    }
15 }

```

6.3 cantor expansion

```

1 int factorial[MAXN];
2 void init(){
3     factorial[0]=1;
4     for(int i=1; i<=MAXN; ++i)factorial[i]=
factorial[i-1]*i;
5 }
6 int encode(const vector<int> &s){
7     int n=s.size(), res=0;
8     for(int i=0; i<n; ++i){
9         int t=0;
10        for(int j=i+1; j<n; ++j)
11            if(s[j]<s[i])++t;

```

```

12        res+=t*factorial[n-i-1];
13    }
14    return res;
15 }
16 vector<int> decode(int a, int n){
17     vector<int> res;
18     vector<bool> vis(n, 0);
19     for(int i=n-1; i>=0; --i){
20         int t=a/factorial[i];
21         for(j=0; j<n; ++j)
22             if(!vis[j]){
23                 if(t==0)break;
24                 --t;
25             }
26         res.push_back(j);
27         vis[j]=1;
28         a%=factorial[i];
29     }
30     return res;
31 }

```

6.4 FFT

```

1 template<typename T, typename VT=vector<
complex<T> > >
2 struct FFT{
3     const T pi;
4     FFT(const T pi=acos((T)-1)):pi(pi){}
5     unsigned bit_reverse(unsigned a, int len){
6         a=((a&0x55555555U)<<1)|((a&0xAAAAAAAAU)>>1);
7         a=((a&0x33333333U)<<2)|((a&0xCCCCCCCCU)>>2);
8         a=((a&0x0F0F0F0FU)<<4)|((a&0xFF0F0F0FU)>>4);
9         a=((a&0x00FF00FFU)<<8)|((a&0xFFFF0000U)>>8);
10        a=((a&0x0000FFFFU)<<16)|((a&0xFFFF0000U)
>>16);
11        return a>>(32-len);
12    }
13    void fft(bool is_inv, VT &in, VT &out, int N)
{
14        int bitlen=__lg(N), num=is_inv?-1:1;
15        for(int i=0; i<N; ++i)out[bit_reverse(i,
bitlen)]=in[i];
16        for(int step=2; step<=N; step<<=1){
17            const int mh=step>>1;
18            for(int i=0; i<mh; ++i){
19                complex<T> wi=exp(complex<T>(0, i*num
*pi/mh));
20                for(int j=i; j<N; j+=step){
21                    int k=j+mh;
22                    complex<T> u=out[j], t=wi*out[k];
23                    out[j]=u+t;
24                    out[k]=u-t;
25                }
26            }
27        }
28        if(is_inv)for(int i=0; i<N; ++i)out[i]/=N;
29    }
30 };

```

6.5 FFT2

```

1 using cd = complex<double>;
2
3 const double PI = acos(-1);
4
5 void fft(vector<cd>& a, bool invert) {
6     int n = a.size();
7
8     // Bit reversal
9     for(int i = 1, j = 0; i < n; i++) {
10         int bit = n >> 1;
11
12         for(; j & bit; bit >>= 1) j ^= bit;
13
14         j ^= bit;
15
16         if(i < j) swap(a[i], a[j]);
17     }
18
19     for(int len = 2; len <= n; len <= 1){
20         double ang = 2 * PI / len * (invert
21             ? -1 : 1);
22         cd wlen(cos(ang), sin(ang));
23
24         for(int i = 0; i < n; i += len){
25             cd w(1);
26
27             for(int j = 0; j < len / 2; j++){
28                 cd u = a[i + j];
29                 cd v = a[i + j + len / 2] *
30                     w;
31                 a[i + j] = u + v;
32                 a[i + j + len / 2] = u - v;
33                 w *= wlen;
34             }
35         }
36     }
37
38     if(invert){
39         for(cd& x : a) x /= n;
40     }
41 }
42 // (a[0] + a[1]x + a[2]x^2 + ...) * (b[0] +
43 // b[1]x + b[2]x^2 + ...)
44 vector<long long> multiply(vector<long long>
45     a, vector<long long> b) {
46     int n = 1;
47
48     while(n < a.size() + b.size()) n <= 1;
49
50     vector<cd> fa(a.begin(), a.end());
51     vector<cd> fb(b.begin(), b.end());
52
53     fa.resize(n);
54     fb.resize(n);
55
56     fft(fa, false);
57     fft(fb, false);
58
59     for(int i = 0; i < n; i++) fa[i] *= fb[i];
60
61     fft(fa, true);

```

6.6 find real root

```

1 // an*x^n + ... + a1x + a0 = 0;
2 int sign(double x){
3     return x < -eps ? -1 : x > eps;
4 }
5
6 double get(const vector<double>&coef, double
7     x){
8     double e = 1, s = 0;
9     for(auto i : coef) s += i*e, e *= x;
10    return s;
11 }
12
13 double find(const vector<double>&coef, int n
14     , double lo, double hi){
15     double sign_lo, sign_hi;
16     if( !(sign_lo = sign(get(coef,lo))) )
17         return lo;
18     if( !(sign_hi = sign(get(coef,hi))) )
19         return hi;
20     if(sign_lo * sign_hi > 0) return INF;
21     for(int stp = 0; stp < 100 && hi - lo >
22         eps; ++stp){
23         double m = (lo+hi)/2.0;
24         int sign_mid = sign(get(coef,m));
25         if(!sign_mid) return m;
26         if(sign_lo*sign_mid < 0) hi = m;
27         else lo = m;
28     }
29     return (lo+hi)/2.0;
30 }
31
32 vector<double> cal(vector<double>coef, int n
33 ) {
34     vector<double>res;
35     if(n == 1){
36         if(sign(coef[1])) res.pb(-coef[0]/coef
37             [1]);
38         return res;
39     }
40     vector<double>dcoef(n);
41     for(int i = 0; i < n; ++i) dcoef[i] = coef
42         [i+1]*(i+1);
43     vector<double>droot = cal(dcoef, n-1);
44     droot.insert(droot.begin(), -INF);
45     droot.pb(INF);
46     for(int i = 0; i < n; ++i){
47         double tmp = find(coef, n, droot[i],
48             droot[i+1]);
49         if(tmp < INF) res.pb(tmp);
50     }
51     return res;
52 }

```

```

45 int main () {
46     vector<double>ve;
47     vector<double>ans = cal(ve, n);
48     // 視情況把答案 +eps · 避免 -0
49 }

```

6.7 FWT

```

1 vector<int> F_OR_T(vector<int> f, bool
2     inverse){
3     for(int i=0; (2<<i)<=f.size(); ++i)
4         for(int j=0; j<f.size(); j+=2<<i)
5             for(int k=0; k<(1<<i); ++k)
6                 f[j+k+(1<<i)] += f[j+k]*(inverse
7                     ? -1:1);
8     return f;
9 }
10 vector<int> rev(vector<int> A) {
11     for(int i=0; i<A.size(); i+=2)
12         swap(A[i],A[i^A.size()-1]);
13     return A;
14 }
15 vector<int> F_AND_T(vector<int> f, bool
16     inverse){
17     return rev(F_OR_T(rev(f), inverse));
18 }
19 vector<int> F_XOR_T(vector<int> f, bool
20     inverse){
21     for(int i=0; (2<<i)<=f.size(); ++i)
22         for(int j=0; j<f.size(); j+=2<<i)
23             for(int k=0; k<(1<<i); ++k){
24                 int u=f[j+k], v=f[j+k+(1<<i)];
25                 f[j+k+(1<<i)] = u-v, f[j+k] = u+v;
26             }
27     if(inverse) for(auto &a:f) a/=f.size();
28     return f;
29 }

```

6.8 LinearCongruence

```

1 pair<LL,LL> LinearCongruence(LL a[],LL b[],
2     LL m[],int n) {
3     // a[i]*x = b[i] ( mod m[i] )
4     for(int i=0;i<n;++i) {
5         LL x, y, d = extgcd(a[i],m[i],x,y);
6         if(b[i]%d!=0) return make_pair(-1LL,0LL);
7         m[i] /= d;
8         b[i] = LLmul(b[i]/d,x,m[i]);
9     }
10    LL lastb = b[0], lastm = m[0];
11    for(int i=1;i<n;++i) {
12        LL x, y, d = extgcd(m[i],lastm,x,y);
13        if((lastb-b[i])%d!=0) return make_pair
14            (-1LL,0LL);
15        lastb = LLmul((lastb-b[i])/d,x,(lastm/d)
16            )*m[i];
17        lastm = (lastm/d)*m[i];
18        lastb = (lastb+b[i])%lastm;
19    }

```

```

16 }
17 return make_pair(lastb<0?lastb+lastm:lastb
18     ,lastm);
19 }

```

6.9 Lucas

```

1 ll C(ll n, ll m, ll p){ // n!/m!/(n-m)!
2     if(n<m) return 0;
3     return f[n]*inv(f[m],p)%p*inv(f[n-m],p)%p;
4 }
5 ll L(ll n, ll m, ll p){
6     if(!m) return 1;
7     return C(n%p,m%p,p)*L(n/p,m/p,p)%p;
8 }
9 ll Wilson(ll n, ll p){ // n!%p
10    if(!n)return 1;
11    ll res=Wilson(n/p, p);
12    if((n/p)%2) return res*(p-f[n%p])%p;
13    return res*f[n%p]%p; // (p-1)!%p=-1
14 }

```

6.10 Matrix

```

1 template<typename T>
2 struct Matrix{
3     using rt = std::vector<T>;
4     using mt = std::vector<rt>;
5     using matrix = Matrix<T>;
6     int r,c;
7     mt m;
8     Matrix(int r,int c):r(r),c(c),m(r,rt(c)){}
9     rt& operator[](int i){return m[i];}
10    matrix operator+(const matrix &a){
11        matrix rev(r,c);
12        for(int i=0;i<r;++i)
13            for(int j=0;j<c;++j)
14                rev[i][j]=m[i][j]+a.m[i][j];
15        return rev;
16    }
17    matrix operator-(const matrix &a){
18        matrix rev(r,c);
19        for(int i=0;i<r;++i)
20            for(int j=0;j<c;++j)
21                rev[i][j]=m[i][j]-a.m[i][j];
22        return rev;
23    }
24    matrix operator*(const matrix &a){
25        matrix rev(r,a.c);
26        matrix tmp(a.c,a.r);
27        for(int i=0;i<a.r;++i)
28            for(int j=0;j<a.c;++j)
29                tmp[j][i]=a.m[i][j];
30        for(int i=0;i<r;++i)
31            for(int j=0;j<a.c;++j)
32                for(int k=0;k<c;++k)
33                    rev.m[i][j]+=m[i][k]*tmp[j][k];
34        return rev;
35    }
36    bool inverse(){

```

```

37 Matrix t(r,r+c);
38 for(int y=0;y<r;y++){
39     t.m[y][c+y] = 1;
40     for(int x=0;x<c;++x)
41         t.m[y][x]=m[y][x];
42 }
43 if( !t.gas() )
44     return false;
45 for(int y=0;y<r;y++)
46     for(int x=0;x<c;++x)
47         m[y][x]=t.m[y][c+x]/t.m[y][y];
48 return true;
49 }
50 T gas(){
51     vector<T> lazy(r,1);
52     bool sign=false;
53     for(int i=0;i<r;++i){
54         if( m[i][i]==0 ){
55             int j=i+1;
56             while(j<r&&!m[j][i])j++;
57             if(j==r)continue;
58             m[i].swap(m[j]);
59             sign=!sign;
60         }
61         for(int j=0;j<r;++j){
62             if(i==j)continue;
63             lazy[j]=lazy[j]*m[i][i];
64             T mx=m[j][i];
65             for(int k=0;k<c;++k)
66                 m[j][k]=m[j][k]*m[i][i]-m[i][k]*mx;
67         }
68     }
69     T det=sign?-1:1;
70     for(int i=0;i<r;++i){
71         det = det*m[i][i];
72         det = det/lazy[i];
73         for(auto &j:m[i])j/=lazy[i];
74     }
75     return det;
76 }
77 };

```

6.11 MillerRobin

```

1 ULL LLMul(ULL a, ULL b, const ULL &mod) {
2     LL ans=0;
3     while(b) {
4         if(b&1) {
5             ans+=a;
6             if(ans>=mod) ans-=mod;
7         }
8         a<<=1, b>>=1;
9         if(a>=mod) a-=mod;
10    }
11    return ans;
12 }
13 ULL mod_mul(ULL a,ULL b,ULL m){
14     a%=m,b%=m; /* fast for m < 2^58 */
15     ULL y=(ULL)((double)a*b/m+0.5);
16     ULL r=(a*b-y*m)%m;
17     return r<0?r+m:r;
18 }

```

```

19 template<typename T>
20 T pow(T a, T b, T mod){ //a^b%mod
21     T ans=1;
22     for(;b;a=mod_mul(a,a,mod),b>>=1)
23         if(b&1)ans=mod_mul(ans,a,mod);
24     return ans;
25 }
26 int sprp[3]={2,7,61}; //int範圍可解
27 int llsprp
28 [7]={2,325,9375,28178,450775,9780504,
29 1795265022}; //至少unsigned Long Long範圍
30 template<typename T>
31 bool isprime(T n,int *sprp,int num){
32     if(n==2)return 1;
33     if(n<2||n%2==0)return 0;
34     int t=0;
35     T u=n-1;
36     for(;u%2==0;++t)u>>=1;
37     for(int i=0;i<num;++i){
38         T a=sprp[i]%n;
39         if(a==0||a==1||a==n-1)continue;
40         T x=pow(a,u,n);
41         if(x==1||x==n-1)continue;
42         for(int j=0;j<t;++j){
43             x=mod_mul(x,x,n);
44             if(x==1)return 0;
45             if(x==n-1)break;
46         }
47         if(x==n-1)continue;
48         return 0;
49     }
50     return 1;
51 }

```

6.12 NTT

```

1 2615053605667*(2^18)+1,3
2 15*(2^27)+1,31
3 479*(2^21)+1,3
4 7*17*(2^23)+1,3
5 3*3*211*(2^19)+1,5
6 25*(2^22)+1,3
7 template<typename T,typename VT=vector<T> >
8 struct NTT{
9     const T P,G;
10    NTT(T p=(1<<23)*7*17+1,T g=3):P(p),G(g){}
11    unsigned bit_reverse(unsigned a,int len){
12        //Look FFT.cpp
13    }
14    T pow_mod(T n,T k,T m){
15        T ans=1;
16        for(n=(n>=m?n%m:n);k;k>>=1){
17            if(k&1)ans=ans*n%m;
18            n=n*n%m;
19        }
20        return ans;
21    }
22    void ntt(bool is_inv,VT &in,VT &out,int N)
23    {
24        int bitlen=__lg(N);
25        for(int i=0;i<N;++i)out[bit_reverse(i,
26            bitlen)]=in[i];

```

```

25 for(int step=2,id=1;step<=N;step<=1,++
26     id){
27     T wn=pow_mod(G,(P-1)>>id,P),wi=1,u,t;
28     const int mh=step>>1;
29     for(int i=0;i<mh;++i){
30         for(int j=i;j<N;j+=step){
31             u=out[j],t=wi*out[j+mh]%P;
32             out[j]=u+t;
33             out[j+mh]=u-t;
34             if(out[j]>=P)out[j]-=P;
35             if(out[j+mh]<0)out[j+mh]+=P;
36         }
37         wi=wi*wn%P;
38     }
39     if(is_inv){
40         for(int i=1;i<N/2;++i)swap(out[i],out[
41             N-i]);
42         T invn=pow_mod(N,P-2,P);
43         for(int i=0;i<N;++i)out[i]=out[i]*invn
44             %P;
45     }
46 }

```

6.13 Simpson

```

1 double simpson(double a,double b){
2     double c=a+(b-a)/2;
3     return (F(a)+4*F(c)+F(b))*(b-a)/6;
4 }
5 double asr(double a,double b,double eps,
6     double A){
7     double c=a+(b-a)/2;
8     double L=simpson(a,c),R=simpson(c,b);
9     if( abs(L+R-A)<15*eps )
10        return L+R+(L+R-A)/15.0;
11    return asr(a,c,eps/2,L)+asr(c,b,eps/2,R);
12 }
13 double asr(double a,double b,double eps){
14     return asr(a,b,eps,simpson(a,b));
15 }

```

6.14 外星模運算

```

1 //a[0]^a[1]^a[2]^...
2 #define maxn 1000000
3 int euler[maxn+5];
4 bool is_prime[maxn+5];
5 void init_euler(){
6     is_prime[1]=1; //一不是質數
7     for(int i=1;i<=maxn;i++)euler[i]=i;
8     for(int i=2;i<=maxn;i++){
9         if(!is_prime[i]){ //是質數
10             euler[i]--;
11             for(int j=i<=1;j<=maxn;j+=i){
12                 is_prime[j]=1;
13                 euler[j]=euler[j]/i*(i-1);
14             }

```

```

15     }
16 }
17 LL pow(LL a,LL b,LL mod){ //a^b%mod
18     LL ans=1;
19     for(;b;a=a*a%mod,b>>=1)
20         if(b&1)ans=ans*a%mod;
21     return ans;
22 }
23 bool isless(LL *a,int n,int k){
24     if(*a==1)return k>1;
25     if(--n==0)return *a<k;
26     int next=0;
27     for(LL b=1;b<k;++next)
28         b*=*a;
29     return isless(a+1,n,next);
30 }
31 LL high_pow(LL *a,int n,LL mod){
32     if(*a==1||--n==0)return *a%mod;
33     int k=0,r=euler[mod];
34     for(LL tma=1;tma!=pow(*a,k+r,mod);++k)
35         tma=tma*(*a)%mod;
36     if(isless(a+1,n,k))return pow(*a,high_pow(
37         a+1,n,k,mod));
38     int tmd=high_pow(a+1,n,r), t=(tmd-k+r)%r;
39     return pow(*a,k+t,mod);
40 }
41 LL a[1000005];
42 int t,mod;
43 int main(){
44     init_euler();
45     scanf("%d",&t);
46     #define n 4
47     while(t--){
48         for(int i=0;i<n;++i)scanf("%lld",&a[i]);
49         scanf("%d",&mod);
50         printf("%lld\n",high_pow(a,n,mod));
51     }
52     return 0;
53 }

```

6.15 數位統計

```

1 ll d[65], dp[65][2]; //up區間是不是完整
2 ll dfs(int p,bool is8,bool up){
3     if(!p)return 1; // 回傳0是不是答案
4     if(!up&&dp[p][is8])return dp[p][is8];
5     int mx = up?d[p]:9; //可以用的有那些
6     ll ans=0;
7     for(int i=0;i<=mx;++i){
8         if( is8&&i==7 )continue;
9         ans += dfs(p-1,i==8,up&&i==mx);
10    }
11    if(!up)dp[p][is8]=ans;
12    return ans;
13 }
14 ll f(ll N){
15     int k=0;
16     while(N){ // 把數字先分解到陣列
17         d[++k] = N%10;
18         N/=10;
19     }

```

```

20 | return dfs(k,false,true);
21 | }

```

6.16 質因數分解

```

1 | LL func(const LL n,const LL mod,const int c)
2 | {
3 |     return (LLmul(n,n,mod)+c+mod)%mod;
4 | }
5 | LL pollorrho(const LL n, const int c) { //循環節長度
6 |     LL a=1, b=1;
7 |     a=func(a,n,c)%n;
8 |     b=func(b,n,c)%n; b=func(b,n,c)%n;
9 |     while(gcd(abs(a-b),n)!=1) {
10 |         a=func(a,n,c)%n;
11 |         b=func(b,n,c)%n; b=func(b,n,c)%n;
12 |     }
13 |     return gcd(abs(a-b),n);
14 | }
15 |
16 | void prefactor(LL &n, vector<LL> &v) {
17 |     for(int i=0;i<12;++i) {
18 |         while(n%prime[i]==0) {
19 |             v.push_back(prime[i]);
20 |             n/=prime[i];
21 |         }
22 |     }
23 | }
24 |
25 | void smallfactor(LL n, vector<LL> &v) {
26 |     if(n<MAXPRIME) {
27 |         while(isp[(int)n]) {
28 |             v.push_back(isp[(int)n]);
29 |             n/=isp[(int)n];
30 |         }
31 |         v.push_back(n);
32 |     } else {
33 |         for(int i=0;i<primecnt&&prime[i]*prime[i]<=n;++i) {
34 |             while(n%prime[i]==0) {
35 |                 v.push_back(prime[i]);
36 |                 n/=prime[i];
37 |             }
38 |         }
39 |         if(n!=1) v.push_back(n);
40 |     }
41 | }
42 |
43 | void comfactor(const LL &n, vector<LL> &v) {
44 |     if(n<1e9) {
45 |         smallfactor(n,v);
46 |         return;
47 |     }
48 |     if(Isprime(n)) {
49 |         v.push_back(n);
50 |         return;
51 |     }
52 |     LL d;
53 |     for(int c=3;;++c) {
54 |         d = pollorrho(n,c);

```

```

55 |         if(d!=n) break;
56 |     }
57 |     comfactor(d,v);
58 |     comfactor(n/d,v);
59 | }
60 |
61 | void Factor(const LL &x, vector<LL> &v) {
62 |     LL n = x;
63 |     if(n==1) { puts("Factor 1"); return; }
64 |     prefactor(n,v);
65 |     if(n==1) return;
66 |     comfactor(n,v);
67 |     sort(v.begin(),v.end());
68 | }
69 |
70 | void AllFactor(const LL &n,vector<LL> &v) {
71 |     vector<LL> tmp;
72 |     Factor(n,tmp);
73 |     v.clear();
74 |     v.push_back(1);
75 |     int len;
76 |     LL now=1;
77 |     for(int i=0;i<tmp.size();++i) {
78 |         if(i==0 || tmp[i]!=tmp[i-1]) {
79 |             len = v.size();
80 |             now = 1;
81 |         }
82 |         now*=tmp[i];
83 |         for(int j=0;j<len;++j)
84 |             v.push_back(v[j]*now);
85 |     }
86 | }

```

7 String

7.1 ac 自動機

```

1 | int trie[(int)5e5 + 5][26], node_cnt = 0,
2 | fail[(int)5e5 + 5]; // ac_automachine
3 | int pattern_end[(int)5e5 + 5];
4 | void insert(string s, int id)
5 | {
6 |     int u = 0;
7 |     for(char c : s){
8 |         int v = c - 'a';
9 |         if(!trie[u][v]) trie[u][v] = ++
10 |             node_cnt;
11 |         u = trie[u][v];
12 |     }
13 |     pattern_end[id] = u;
14 | }
15 |
16 | void build_ac_automachine()
17 | {
18 |     //bfs
19 |     queue<int> q;
20 |     for(int i = 0; i < 26; i++) if(trie[0][i]
21 |         > 0) q.push(trie[0][i]);
22 |     while(q.size()){
23 |         int x = q.front(); q.pop();
24 |         for(int i = 0; i < 26; i++){
25 |             if(trie[x][i] > 0){

```

```

22 |         fail[trie[x][i]] = trie[fail
23 |             [x]][i];
24 |         q.push(trie[x][i]);
25 |     } else trie[x][i] = trie[fail[x]][
26 |         i];
27 |     }
28 | }

```

7.2 hash

```

1 | const int a = 31;
2 | const int mod1 = 1e9 + 7;
3 | const int mod2 = 1e9 + 9;
4 | struct BIT{
5 |     int n;
6 |     vector<int> bit1, bit2;
7 |     BIT(int _n){
8 |         n = _n;
9 |         bit1.resize(n + 1, 0);
10 |         bit2.resize(n + 1, 0);
11 |     }
12 |     void add(int x, int val1, int val2){
13 |         int y = x;
14 |         for(; x <= n; x += x & -x) bit1[x] =
15 |             (bit1[x] + val1) % mod1;
16 |         for(; y <= n; y += y & -y) bit2[y] =
17 |             (bit2[y] + val2) % mod2;
18 |     }
19 |     int que1(int x){
20 |         int ans = 0;
21 |         for(; x; x -= x & -x) ans = (ans +
22 |             bit1[x]) % mod1;
23 |         return ans;
24 |     }
25 |     int que2(int x){
26 |         int ans = 0;
27 |         for(; x; x -= x & -x) ans = (ans +
28 |             bit2[x]) % mod2;
29 |         return ans;
30 |     }
31 |     int query1(int l, int r){
32 |         return (que1(r) - que1(l - 1) + mod1
33 |             ) % mod1;
34 |     }
35 |     int query2(int l, int r){
36 |         return (que2(r) - que2(l - 1) + mod2
37 |             ) % mod2;
38 |     }
39 | };
40 | void solve()
41 | {
42 |     string s;
43 |     cin >> s;
44 |     int n = s.size();
45 |     vector<int> f1(n, 1), f2(n, 1);
46 |     BIT bit(n);
47 |     for(int i = 1; i < n; i++) f1[i] = (f1[i
48 |         - 1] * a) % mod1;
49 |     for(int i = 1; i < n; i++) f2[i] = (f2[i
50 |         - 1] * a) % mod2;

```

```

43 |     for(int i = 0; i < n; i++) bit.add(i +
44 |         1, f1[i] * (s[i] - 'a' + 1) % mod1,
45 |         f2[i] * (s[i] - 'a' + 1) % mod2);
46 |     set<pii> cnt;
47 |     for(int i = k; i <= n; i++){
48 |         int sum1 = bit.query1(i - k + 1, i)
49 |             * fpow(f1[i - k], mod1 - 2, mod1)
50 |             % mod1;
51 |         int sum2 = bit.query2(i - k + 1, i)
52 |             * fpow(f2[i - k], mod2 - 2, mod2)
53 |             % mod2;
54 |         cnt.insert({sum1, sum2});
55 |     }
56 |     cout << cnt.size() << "\n";

```

7.3 KMP

```

1 | vector<int> pi(s.size(), 0);
2 | for(int i = 1, j = 0; i < s.size(); i++){
3 |     while(j > 0 and s[i] != s[j]) j = pi[j -
4 |         1];
5 |     if(s[i] == s[j]) j++;
6 |     pi[i] = j;

```

7.4 manacher

```

1 | string s, t = "^";
2 | cin >> s;
3 | for(char c : s){
4 |     t += "#";
5 |     t += c;
6 | }
7 | t += "##";
8 | vector<int> p(t.size(), 0);
9 | int c = 0, r = 0, len = 0, center = 0;
10 | for(int i = 1; i < t.size() - 1; i++){
11 |     if(i < r) p[i] = min(r - i, p[2 * c - i
12 |         ]);
13 |     while(t[i + p[i] + 1] == t[i - p[i] -
14 |         1]) p[i]++;
15 |     if(i + p[i] > r){
16 |         r = i + p[i];
17 |         c = i;
18 |     }
19 |     if(p[i] > len){
20 |         len = p[i];
21 |         center = i;
22 |     }
23 | }
24 | cout << s.substr((center - len) / 2, len) <<
25 |     "\n"; //Longest Palindrome

```

7.5 suffix-array

```

1 vector<int> buildSuffixArray(string s, int n
2 )
3 {
4     vector<int> sa(n), tmp(n), rank(n);
5     /*
6     sa[i]: 從sa[i]開始的後綴字串是第i小
7     rank[i]: 從i開始的後綴字串是第幾小
8     tmp: rank的暫存
9     */
10    for(int i = 0; i < n; i++){ // 初始化長
11        度為1的後綴字串
12        sa[i] = i;
13        rank[i] = s[i] - 'a';
14    }
15    for(int k = 1; k < n; k <= 1){
16        auto cmp = [&](int i, int j){ //比較
17            長度為2*k的字串
18            if(rank[i] != rank[j]) return
19                rank[i] < rank[j];
20            int ri = i + k < n ? rank[i + k]
21                : -1;
22            int rj = j + k < n ? rank[j + k]
23                : -1;
24            return ri < rj;
25        };
26        sort(sa.begin(), sa.end(), cmp); //
27        sa 按照字典序(rank)排列
28        /*
29        更新rank
30        要先用tmp存，因為cmp會用到rank的舊資
31        料
32        */
33        tmp[sa[0]] = 0;
34        for(int i = 1; i < n; i++) tmp[sa[i]
35            ] = tmp[sa[i - 1]] + (cmp(sa[i]
36            - 1], sa[i]) ? 1 : 0);
37        rank = tmp;
38    }
39    return sa;
40 }
41 vector<int> buildLCP(string s, vector<int>
42     sa, int n)
43 {
44     vector<int> rank(n, 0); // rank[i]: 從s[
45         i]開始的字尾字串按照字典序排序後的位置
46     for(int i = 0; i < n; i++) rank[sa[i]] =
47         i;
48     int h = 0; // 因為是按字串s原始位置順序計
49         算，用前綴相似的特性，h每次最多只會
50         減少1
51     vector<int> lcp(n, 0); // 與字典序前一個
52         的字尾字串的最長共同前綴長度
53     for(int i = 0; i < n; i++){
54         if(rank[i] > 0){
55             int j = sa[rank[i] - 1]; // 字典
56                 序前一個的字尾字串
57             while(i + h < n and j + h < n
58                 and s[i + h] == s[j + h]) h
59                 ++;
60             lcp[rank[i]] = h;
61             if(h > 0) h--;
62         }
63     }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
101 }
102 }
103 }
104 }
105 }
106 }
107 }
108 }
109 }
110 }
111 }
112 }
113 }
114 }
115 }
116 }
117 }
118 }
119 }
120 }
121 }
122 }
123 }
124 }
125 }
126 }
127 }
128 }
129 }
130 }
131 }
132 }
133 }
134 }
135 }
136 }
137 }
138 }
139 }
140 }
141 }
142 }
143 }
144 }
145 }
146 }
147 }
148 }
149 }
150 }
151 }
152 }
153 }
154 }
155 }
156 }
157 }
158 }
159 }
160 }
161 }
162 }
163 }
164 }
165 }
166 }
167 }
168 }
169 }
170 }
171 }
172 }
173 }
174 }
175 }
176 }
177 }
178 }
179 }
180 }
181 }
182 }
183 }
184 }
185 }
186 }
187 }
188 }
189 }
190 }
191 }
192 }
193 }
194 }
195 }
196 }
197 }
198 }
199 }
200 }
201 }
202 }
203 }
204 }
205 }
206 }
207 }
208 }
209 }
210 }
211 }
212 }
213 }
214 }
215 }
216 }
217 }
218 }
219 }
220 }
221 }
222 }
223 }
224 }
225 }
226 }
227 }
228 }
229 }
230 }
231 }
232 }
233 }
234 }
235 }
236 }
237 }
238 }
239 }
240 }
241 }
242 }
243 }
244 }
245 }
246 }
247 }
248 }
249 }
250 }
251 }
252 }
253 }
254 }
255 }
256 }
257 }
258 }
259 }
260 }
261 }
262 }
263 }
264 }
265 }
266 }
267 }
268 }
269 }
270 }
271 }
272 }
273 }
274 }
275 }
276 }
277 }
278 }
279 }
280 }
281 }
282 }
283 }
284 }
285 }
286 }
287 }
288 }
289 }
290 }
291 }
292 }
293 }
294 }
295 }
296 }
297 }
298 }
299 }
300 }
301 }
302 }
303 }
304 }
305 }
306 }
307 }
308 }
309 }
310 }
311 }
312 }
313 }
314 }
315 }
316 }
317 }
318 }
319 }
320 }
321 }
322 }
323 }
324 }
325 }
326 }
327 }
328 }
329 }
330 }
331 }
332 }
333 }
334 }
335 }
336 }
337 }
338 }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }

```

7.6 Zfunction

```

1 vector<int> ZValue(const string &s) {
2     vector<int> z(s.length(), 0);
3     for (int i = 1, l = 1, r = 1; i < (int)
4         s.length(); i++) {
5         if (i < r) {
6             z[i] = min(z[i - 1], r - i);
7         }
8         while (i + z[i] < (int) s.length()
9             && s[i + z[i]] == s[z[i]]) {
10             z[i]++;
11         }
12         if (r < i + z[i]) {
13             l = i;
14             r = i + z[i];
15         }
16     }
17     return z;
18 }

```

8 Tarjan

8.1 dominator tree

```

1 struct dominator_tree{
2     static const int MAXN=5005;
3     int n; // 1-base
4     vector<int> G[MAXN], rG[MAXN];
5     int pa[MAXN], dfn[MAXN], id[MAXN], dfnCnt;
6     int semi[MAXN], idom[MAXN], best[MAXN];
7     vector<int> tree[MAXN]; // tree here
8     void init(int _n){
9         n = _n;
10        for(int i=1; i<=n; ++i)
11            G[i].clear(), rG[i].clear();
12    }
13    void add_edge(int u, int v){
14        G[u].push_back(v);
15        rG[v].push_back(u);
16    }
17    void dfs(int u){

```

```

18        id[dfn[u]=++dfnCnt]=u;
19        for(auto v:G[u]) if(!dfn[v])
20            dfs(v), pa[dfn[v]]=dfn[u];
21    }
22    int find(int y, int x){
23        if(y <= x) return y;
24        int tmp = find(pa[y], x);
25        if(semi[best[y]] > semi[best[pa[y]]])
26            best[y] = best[pa[y]];
27        return pa[y] = tmp;
28    }
29    void tarjan(int root){
30        dfnCnt = 0;
31        for(int i=1; i<=n; ++i){
32            dfn[i] = idom[i] = 0;
33            tree[i].clear();
34            best[i] = semi[i] = i;
35        }
36        dfs(root);
37        for(int i=dfnCnt; i>1; --i){
38            int u = id[i];
39            for(auto v:rG[u]) if(v=dfn[v]){
40                find(v, i);
41                semi[i]=min(semi[i], semi[best[v]]);
42            }
43            tree[semi[i]].push_back(i);
44            for(auto v:tree[pa[i]]){
45                find(v, pa[i]);
46                idom[v] = semi[best[v]]==pa[i]
47                    ? pa[i] : best[v];
48            }
49            tree[pa[i]].clear();
50        }
51        for(int i=2; i<=dfnCnt; ++i){
52            if(idom[i] != semi[i])
53                idom[i] = idom[idom[i]];
54            tree[id[idom[i]]].push_back(id[i]);
55        }
56    }
57 } dom;

```

8.2 tnfshb017 2 sat

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define MAXN 8001
4 #define MAXN2 MAXN*4
5 #define n(X) ((X)+2*N)
6 vector<int> v[MAXN2], rv[MAXN2], vis_t;
7 int N,M;
8 void addedge(int s,int e){
9     v[s].push_back(e);
10    rv[e].push_back(s);
11 }
12 int scc[MAXN2];
13 bool vis[MAXN2]={false};
14 void dfs(vector<int> *uv,int n,int k=-1){
15     vis[n]=true;
16     for(int i=0;i<uv[n].size();++i)
17         if(!vis[uv[n][i]])
18             dfs(uv,uv[n][i],k);
19     if(uv==v)vis_t.push_back(n);
20     scc[n]=k;

```

```

21 }
22 void solve(){
23     for(int i=1;i<=N;++i){
24         if(!vis[i])dfs(v,i);
25         if(!vis[n(i)])dfs(v,n(i));
26     }
27     memset(vis,0,sizeof(vis));
28     int c=0;
29     for(int i=vis_t.size()-1;i>=0;--i)
30         if(!vis[vis_t[i]])
31             dfs(rv,vis_t[i],c++);
32 }
33 int main(){
34     int a,b;
35     scanf("%d%d",&N,&M);
36     for(int i=1;i<=N;++i){
37         // (A or B)&(!A & !B) A^B
38         a=i*2-1;
39         b=i*2;
40         addedge(n(a),b);
41         addedge(n(b),a);
42         addedge(a,n(b));
43         addedge(b,n(a));
44     }
45     while(M--){
46         scanf("%d%d",&a,&b);
47         a = a>0?a*2-1:-a*2;
48         b = b>0?b*2-1:-b*2;
49         // A or B
50         addedge(n(a),b);
51         addedge(n(b),a);
52     }
53     solve();
54     bool check=true;
55     for(int i=1;i<=2*N;++i)
56         if(scc[i]==scc[n(i)])
57             check=false;
58     if(check){
59         printf("%d\n",N);
60         for(int i=1;i<=2*N;i+=2){
61             if(scc[i]>scc[i+2*N]) putchar('+');
62             else putchar('-');
63         }
64         puts("");
65     }else puts("0");
66     return 0;
67 }

```

8.3 橋連通分量

```

1 #define N 1005
2 struct edge{
3     int u,v;
4     bool is_bridge;
5     edge(int u=0,int v=0):u(u),v(v),is_bridge
6         (0){}
7 };
8 vector<edge> E;
9 vector<int> G[N]; // 1-base
10 int low[N],vis[N],Time;
11 int bcc_id[N],bridge_cnt,bcc_cnt; // 1-base
12 int st[N],top; // BCC用
13 void add_edge(int u,int v){

```

```

13 G[u].push_back(E.size());
14 E.emplace_back(u,v);
15 G[v].push_back(E.size());
16 E.emplace_back(v,u);
17 }
18 void dfs(int u,int re=-1){//u當前點·re為u連
    接前一個點的邊
19 int v;
20 low[u]=vis[u]=++Time;
21 st[top++]=u;
22 for(int e:G[u]){
23     v=E[e].v;
24     if(!vis[v]){
25         dfs(v,e^1);//e^1反向邊
26         low[u]=min(low[u],low[v]);
27         if(vis[u]<low[v]){
28             E[e].is_bridge=E[e^1].is_bridge=1;
29             ++bridge_cnt;
30         }
31     }else if(vis[v]<vis[u]&&v!=re){
32         low[u]=min(low[u],vis[v]);
33     }
34     if(vis[u]==low[u]){//處理BCC
35         ++bcc_cnt;//1-base
36         do bcc_id[v=st[--top]]=bcc_cnt;//每個點
           所在的BCC
37         while(v!=u);
38     }
39 }
40 void bcc_init(int n){
41     Time=bcc_cnt=bridge_cnt=top=0;
42     E.clear();
43     for(int i=1;i<=n;++i){
44         G[i].clear();
45         vis[i]=bcc_id[i]=0;
46     }
47 }

```

8.4 雙連通分量 & 割點

```

1 #define N 1005
2 vector<int> G[N];// 1-base
3 vector<int> bcc[N];//存每塊雙連通分量的點
4 int low[N],vis[N],Time;
5 int bcc_id[N],bcc_cnt;// 1-base
6 bool is_cut[N];//是否為割點
7 int st[N],top;
8 void dfs(int u,int pa=-1){//u當前點·pa父親
9     int t, child=0;
10    low[u]=vis[u]=++Time;
11    st[top++]=u;
12    for(int v:G[u]){
13        if(!vis[v]){
14            dfs(v,u),++child;
15            low[u]=min(low[u],low[v]);
16            if(vis[u]<=low[v]){
17                is_cut[u]=1;
18                bcc[++bcc_cnt].clear();
19                do{
20                    bcc_id[t=st[--top]]=bcc_cnt;
21                    bcc[bcc_cnt].push_back(t);

```

```

22        }while(t!=v);
23        bcc_id[u]=bcc_cnt;
24        bcc[bcc_cnt].push_back(u);
25    }
26    }else if(vis[v]<vis[u]&&v!=pa){//反向邊
27        low[u] = min(low[u],vis[v]);
28    }//u是dfs樹的根要特判
29    if(pa==-1&&child<2)is_cut[u]=0;
30 }
31 void bcc_init(int n){
32     Time=bcc_cnt=top=0;
33     for(int i=1;i<=n;++i){
34         G[i].clear();
35         is_cut[i]=vis[i]=bcc_id[i]=0;
36     }
37 }

```

9 Tree Problem

9.1 centroid decomposition

1 Given a tree of n nodes, your task is to count the number of distinct paths that have at least k_1 and at most k_2 edges.

2 Input

3 The first input line contains three integers n , k_1 and k_2 : the number of nodes and the path lengths. The nodes are numbered $1, 2, \dots, n$.

4 Then there are $n-1$ lines describing the edges. Each line contains two integers a and b : there is an edge between nodes a and b .

5 Output

6 Print one integer: the number of paths.

7 Constraints

8 $1 \leq k_1 \leq k_2 \leq n \leq 2 \cdot 10^5$

9 $1 \leq a, b \leq n$

10 Example

11 Input:

```

12 5 2 3
13 1 2
14 2 3
15 3 4
16 3 5

```

17 Output:

```

18 6

```

19 #include<bits/stdc++.h>

20 using namespace std;

21 #pragma GCC optimize("Ofast")

22 #define KCC ios::sync_with_stdio(0);cin.tie(0);

23 #define pb push_back

24 #define pii pair<long long, long long>

25 #define F first

26 #define S second

27 // #define int long long

```

31 #define MAXN (int)2e5 + 5
32
33 struct BIT{
34     int n;
35     vector<long long> bit;
36     void initial(int _n){
37         n = _n;
38         bit.resize(n + 5, 0);
39     }
40     void upd(int x, long long val){
41         x++;
42         for(; x <= n; x += x & -x) bit[x] += val;
43     }
44     long long query(int x){
45         x++;
46         long long ans = 0;
47         for(; x > 0; x -= x & -x) ans += bit[x];
48         return ans;
49     }
50 }bit;
51
52 int n, k1, k2;
53 vector<int> g[MAXN];
54 bool removed[MAXN];
55 int sz[MAXN];
56 long long ans = 0;
57 void dfs_sz(int x, int p)
58 {
59     sz[x] = 1;
60     for(int i : g[x]){
61         if(i == p or removed[i]) continue;
62         dfs_sz(i, x);
63         sz[x] += sz[i];
64     }
65 }
66 int dfs_centroid(int x, int p, int tot)
67 {
68     for(int i : g[x]){
69         if(i == p or removed[i]) continue;
70         if(sz[i] > (tot >> 1)) return dfs_centroid(i, x, tot);
71     }
72     return x;
73 }
74 void dfs_dis(int x, int p, int d, vector<int> &dis)
75 {
76     if(d > k2) return;
77     dis.pb(d);
78     for(int i : g[x]){
79         if(i == p or removed[i]) continue;
80         dfs_dis(i, x, d + 1, dis);
81     }
82 }
83 vector<int> dis, used, cnt(MAXN, 0);
84 void decompose(int x)
85 {
86     dfs_sz(x, -1);
87     int c = dfs_centroid(x, -1, sz[x]);
88     removed[c] = true;
89     used.clear();
90     bit.upd(0, 1);
91     cnt[0]++;
92     used.pb(0);

```

```

93     for(int i : g[c]){
94         if(removed[i]) continue;
95         dis.clear();
96         dfs_dis(i, c, 1, dis);
97     }
98     for(int d : dis){
99         if(k2 >= d) ans += bit.query(k2 - d) - bit.query(k1 - d - 1);
100     }
101     /*
102     k2 >= d + x >= k1
103     k2 - d >= x >= k1 - d
104     */
105     for(int d : dis){
106         bit.upd(d, 1);
107         cnt[d]++;
108         if(cnt[d] == 1) used.pb(d);
109     }
110 }
111 for(int i : used) bit.upd(i, -cnt[i]);
112 for(int i : used) cnt[i] = 0;
113
114 for(int i : g[c]){
115     if(!removed[i]){
116         decompose(i);
117     }
118 }
119 }
120 signed main()
121 {
122     KCC
123     cin >> n >> k1 >> k2;
124     for(int i = 0; i < n - 1; i++){
125         int u, v; cin >> u >> v;
126         g[u].pb(v);
127         g[v].pb(u);
128     }
129     // cnt.assign(k + 1, 0);
130     bit.initial(k2 + 1);
131     decompose(1);
132     cout << ans << "\n";
133     return 0;
134 }

```

9.2 find-centroid

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define KCC ios::sync_with_stdio(0);cin.tie(0);
5 #define pb push_back
6
7 int subtree[(int)2e5 + 5], n;
8 vector<int> g[(int)2e5 + 5];
9 void dfs(int now, int par)
10 {
11     subtree[now] = 1;
12     for(auto w : g[now]){
13         if(w != par){
14             dfs(w, now);
15             subtree[now] += subtree[w];

```

```

16     }
17 }
18 return ;
19 }
20 int get(int now, int par)
21 {
22     for(auto w : g[now]){
23         if(w != par and subtree[w] * 2 > n)
24             return get(w, now);
25     }
26     return now;
27 }
28 signed main()
29 {
30     KCC
31     cin >> n;
32     for(int i = 0; i < n - 1; i++){
33         int a, b; cin >> a >> b;
34         g[a].pb(b);
35         g[b].pb(a);
36     }
37     dfs(1, 0);
38     cout << get(1, 0) << "\n";
39     return 0;

```

9.3 hld

```

1 vector<int> edge[MAXN];
2 int ar[MAXN], sz[MAXN], par[MAXN], son[MAXN]
  ], dep[MAXN], dfn[MAXN], top[MAXN], t =
  0;
3 void dfs_son(int x, int p, int d)
4 {
5     sz[x] = 1;
6     dep[x] = d;
7     son[x] = 0;
8     for(int i : edge[x]){
9         if(i != p){
10             dfs_son(i, x, d + 1);
11             sz[x] += sz[i];
12             par[i] = x;
13             if(sz[i] > sz[son[x]]) son[x] =
14                 i;
15         }
16     }
17 void dfs_hld(int x, int p)
18 {
19     dfn[x] = ++t;
20     if(son[x] != 0){
21         top[son[x]] = top[x];
22         dfs_hld(son[x], x);
23     }
24     for(int i : edge[x]){
25         if(i != p and i != son[x]){
26             top[i] = i;
27             dfs_hld(i, x);
28         }
29     }
30 }
31 int main()
32 {

```

```

33 int n, q;
34 cin >> n >> q;
35 for(int i = 1; i <= n; i++) cin >> ar[i]
  ];
36 for(int i = 0; i < n - 1; i++){
37     int u, v;
38     cin >> u >> v;
39     edge[u].pb(v);
40     edge[v].pb(u);
41 }
42 dfs_son(1, 0, 1);
43 top[1] = 1;
44 dfs_hld(1, 0);
45
46 vector<int> v(n + 5);
47 for(int i = 1; i <= n; i++) v[dfn[i]] =
48     ar[i];
49 seg_tree seg(n);
50 seg.build(v, 1, n, 1);
51 while(q--){
52     int op; cin >> op;
53     if(op == 1){ //change the value of
54         node s to x
55         int s, x;
56         cin >> s >> x;
57         seg.upd(dfn[s], x, 1, n, 1);
58     }
59     else{ //find the maximum value on
60         the path between nodes a and b.
61         int a, b, ans = 0;
62         cin >> a >> b;
63         while(top[a] != top[b]){
64             if(dfn[top[a]] < dfn[top[b]
65                 ]]) swap(a, b);
66             ans = max(ans, seg.query(dfn
67                 [top[a]], dfn[a], 1, n,
68                 1));
69             a = par[top[a]];
70         }
71         if(dfn[a] > dfn[b]) swap(a, b);
72         ans = max(ans, seg.query(dfn[a],
73             dfn[b], 1, n, 1));
74         cout << ans << " ";
75     }
76 }

```

9.4 lca

```

1 void dfs(int now, int par)
2 {
3     deep[now] = deep[par] + 1;
4     p[now][0] = par;
5     for(int i : v[now]) if(i != par) dfs(i,
6         now);
7     return;
8 }
9 int lca(int u, int v)
10 {
11     int x = deep[u], y = deep[v];
12     if(x > y){
13         swap(u, v);
14     }
15     swap(x, y);

```

```

14 }
15 y -= x; // x < y
16 for(int i = 25; i >= 0; i--) if(y & (1 <<
17     i)) v = p[v][i];
18 if(u == v) return u;
19 for(int i = 25; i >= 0; i--){
20     if(p[u][i] != p[v][i]){
21         u = p[u][i];
22         v = p[v][i];
23     }
24 }
25 return p[u][0];
26 }
27 for(int j = 1; j < 30; j++){
28     for(int i = 2; i <= n; i++){
29         p[i][j] = p[p[i][j - 1]][j - 1];
30     }
31 }

```

10 compare

10.1 check

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int main() {
4     // 1. 在迴圈外編譯，並檢查是否成功
5     cout << "Compiling files..." << endl;
6
7     // system() 回傳 0 代表成功執行
8     int c1 = system("g++ -Wall -std=c++20
9         sol.cpp -o sol.exe");
10    int c2 = system("g++ -Wall -std=c++20 bf
11        .cpp -o bf.exe");
12    int c3 = system("g++ -Wall -std=c++20
13        gen.cpp -o gen.exe");
14
15    if (c1 != 0 || c2 != 0 || c3 != 0) {
16        cout << "Compilation Failed! Check
17            if sol.cpp, bf.cpp, and gen.cpp
18            exist." << endl;
19        return 1;
20    }
21
22    cout << "Compilation successful.
23        Starting stress test..." << endl;
24
25    for (int T = 1; ; T++) {
26        // 2. 執行程序
27        system("gen.exe > input.txt");
28        system("sol.exe < input.txt > sol.
29            out");
30        system("bf.exe < input.txt > bf.out"
31            );
32
33        // 3. 使用 fc /W 比對
34        if (system("fc sol.out bf.out > nul"
35            )) {
36            system("fc sol.out bf.out");
37        }
38    }

```

```

28     cout << "WA on Test #" << T << "
29         !" << endl;
30     // 發生錯誤時停止，此時 input.
31     txt 就是出錯的測資
32     break;
33 } else {
34     if (T % 10 == 0) cout << "Passed
35         " << T << " tests..." <<
36         endl;
37 }
38 }
39 return 0;

```

10.2 gen

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 // 使用隨機種子，確保每次生成的數據都不同
5 mt19937_64 rng(chrono::steady_clock::now().
6     time_since_epoch().count());
7
8 // 生成 [L, r] 範圍內的隨機整數
9 long long rnd(long long l, long long r) {
10     return uniform_int_distribution<long
11         long>(l, r)(rng);
12 }
13
14 // 生成一棵隨機樹 (N 個點 · N-1 條邊)
15 void gen_tree(int n) {
16     for (int i = 2; i <= n; i++) {
17         // 讓每個點 i 連向 [1, i-1] 中的隨機
18         // 點
19         int u = i;
20         int v = rnd(1, i - 1);
21         cout << u << " " << v << "\n";
22     }
23 }

```

11 default

11.1 debug

```

1 #ifndef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr, "%s - %d : (%s) = ",
4         __PRETTY_FUNCTION__, __LINE__, #
5         __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8
9 template<typename I> void _DO(I&&x){cerr<<x
10     <<endl;}
11
12 template<typename I, typename...T> void _DO(I
13     &&x, T&&...tail){cerr<<x<<" "; _DO(tail
14     ...);}
15
16 #else

```

```
9 #define dbg(...)
10 #endif
```

11.2 ext

```
1 #include<bits/extc++.h>
2 #include<ext/pd_ds/assoc_container.hpp>
3 #include<ext/pd_ds/tree_policy.hpp>
4 using namespace __gnu_cxx;
5 using namespace __gnu_pbds;
6 template<typename T>
7 using pbds_set = tree<T, null_type, less<T>,
8     rb_tree_tag,
9     tree_order_statistics_node_update>;
10 template<typename T, typename U>
11 using pbds_map = tree<T, U, less<T>,
12     rb_tree_tag,
13     tree_order_statistics_node_update>;
14 using heap = __gnu_pbds::priority_queue<int>;
15 //s.find_by_order(1); //0 base
16 //s.order_of_key(1);
```

11.3 IncStack

```
1 //Magic
2 #pragma GCC optimize "Ofast"
3 //stack resize, change esp to rsp if 64-bit
4 //system
5 asm("mov %0, %%esp\n" :: "g" (mem+100000000));
6 -Wl,--stack,214748364 -trigraphs
7 #pragma comment(linker, "/STACK
8 :1024000000,1024000000")
9 //linux stack resize
10 #include<sys/resource.h>
11 void increase_stack(){
12     const rlim_t ks=64*1024*1024;
13     struct rlimit rl;
14     int res=getrlimit(RLIMIT_STACK,&rl);
15     if(!res&&rl.rlim_cur<ks){
16         rl.rlim_cur=ks;
17         res=setrlimit(RLIMIT_STACK,&rl);
18     }
19 }
```

11.4 input

```
1 inline int read(){
2     int x=0; bool f=0; char c=getchar();
3     while(ch<'0' || '9'<ch)f|=ch=='- ',ch=getchar();
4     while('0'<=ch&&ch<='9')x=x*10-'0'+ch,ch=
5     getchar();
6     return f?-x:x;
7 }
8 // #!/bin/bash
9 // g++ -std=c++11 -O2 -Wall -Wextra -Wno-
10 unused-result -DDEBUG $1 && ./a.out
```

```
9 // -fsanitize=address -fsanitize=undefined
10 -fsanitize=return
```

12 dp 優化

12.1 2D1D 優化

```
1 /*
2 dp(L,r) = min L≤k<r {dp(L, k) + dp(k + 1, r)
3     + w(L,r)} L ≤ r
4 dp(L,r) = min L≤k<r {dp(L, k) + w(k + 1, r)}
5     L ≤ r
6 以上暴力做會是 O(n^3) · 若 w 滿足四邊形不等式
7 以及區間單調
8 性 · 總體時間複雜度可壓到 O(n^2) ·
9 區間單調性
10 @a, b, c, d, a ≤ b ≤ c ≤ d, w(a, d) ≥ w(b, c)
11 )
12 若 w 滿足四邊形不等式以及區間單調性 · 那麼這
13 個 DP 也
14 滿足四邊形不等式 ·
15 如果 2D/1D DP 滿足四邊形不等式 · 令 p(L,r) 代
16 表 dp(L,r)
17 的最優決策點 · 則有 p(L,r - 1) ≤ p(L,r) ≤ p(L
18 + 1, r)
19 */
20 vector<vector<ll>> dp(n, vector<ll>(n, inf))
21 ;
22 vector<vector<int>> p(n, vector<int>(n));
23 ll w(int l, int r) {} // [L, R]
24 for (int len = 0; len < n; len++) {
25     for (int i = 0; i + len < n; i++) {
26         int j = i + len;
27         if (i == j) {
28             dp[i][j] = 0, p[i][j] = i;
29             continue;
30         }
31         for (int k = p[i][j - 1]; k ≤ min(p
32 [i + 1][j], j - 1); k++) {
33             ll x = dp[i][k] + dp[k + 1][j] +
34 w(i, j);
35             if (x < dp[i][j]) dp[i][j] = x,
36 p[i][j] = k;
37         }
38     }
39 }
40 }
```

12.2 CHT

```
1 // 當放入的直線斜率單調且查詢單調可以使用
2 deque
3 // 否則用李超樹
4 struct Line
```

```
4 {
5     int m, b;
6     int val(int x){
7         return m * x + b;
8     }
9 };
10 bool bad(Line a, Line b, Line c)
11 {
12     // ab交點x座標 >= bc交點x座標
13     // (b.b - a.b) / (a.m - b.m) >= (c.b - b
14     .b) / (b.m - c.m)
15     return (b.b - a.b) * (b.m - c.m) >= (c.b
16     - b.b) * (a.m - b.m);
17 }
18 deque<Line> dq;
19 dq.pb({x, 0});
20 for(int i = 1; i ≤ n; i++){
21     while(dq.size() > 1 and dq[0].val(s[i])
22     >= dq[1].val(s[i])) dq.pop_front();
23     dp[i] = dq[0].val(s[i]);
24     Line tmp = {f[i], dp[i]};
25     while(dq.size() >= 2 and bad(dq[dq.size
26     () - 2], dq.back(), tmp)) dq.
27     pop_back();
28     dq.pb(tmp);
29 }
```

12.3 分治優化

```
1 /*
2 dp(i) = h(i) + min/max j<i {dp(j) + w(j, i)}
3 其中 w(j, i) 必須滿足四邊形不等式 ·
4 四邊形不等式
5 對於任意 a ≤ b ≤ c ≤ d · 若滿足
6 w(a, c) + w(b, d) ≤ (≥) w(a, d) + w(b, c)
7 則稱 w 滿足四邊形不等式 ·
8 */
9 ll w(int l, int r) {} // (L, R]
10 void solve(int j, int l, int r, int optl,
11     int optr) {
12     if (l > r) return;
13     int m = (l + r) / 2;
14     dp[j][m] = INF;
15     int optm = -1;
16     for (int p = optl; p ≤ min(m - 1, optr)
17     ; p++) {
18         ll val = dp[j - 1][p] + w(p, m);
19         if (val < dp[j][m]) {
20             dp[j][m] = val;
21             optm = p;
22         }
23     }
24     solve(j, l, m - 1, optl, optm);
25     solve(j, m + 1, r, optm, optr);
26 }
```

13 other

13.1 WhatDay

```
1 int whatday(int y,int m,int d){
2     if(m<=2)m+=12,--y;
3     if(y<1752||y==1752&&m<9||y==1752&&m==9&&d
4     <3)
5         return (d+2*m+3*(m+1)/5+y+y/4+5)%7;
6     return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)
7     %7;
8 }
```

13.2 上下最大正方形

```
1 void solve(int n,int a[],int b[]){// 1-base
2     int ans=0;
3     deque<int>da,db;
4     for(int l=1,r=1;r<=n;++r){
5         while(da.size()&&a[da.back()]>=a[r]){
6             da.pop_back();
7         }
8         da.push_back(r);
9         while(db.size()&&b[db.back()]>=b[r]){
10             db.pop_back();
11         }
12         db.push_back(r);
13         for(int d=a[da.front()]+b[db.front()];r-
14             1+l>d;++l){
15             if(da.front()==l)da.pop_front();
16             if(db.front()==l)db.pop_front();
17             if(da.size()&&db.size()){
18                 d=a[da.front()]+b[db.front()];
19             }
20             ans=max(ans,r-l+1);
21         }
22     }
23     printf("%d\n",ans);
24 }
```

13.3 最大矩形

```
1 LL max_rectangle(vector<int> s){
2     stack<pair<int,int>> st;
3     st.push(make_pair(-1,0));
4     s.push_back(0);
5     LL ans=0;
6     for(size_t i=0;i<s.size();++i){
7         int h=s[i];
8         pair<int,int> now=make_pair(h,i);
9         while(h<st.top().first){
10             now=st.top();
11             st.pop();
12             ans=max(ans,(LL)(i-now.second)*now.
13                 first);
14         }
15         if(h>st.top().first){
```

```

15     st.push(make_pair(h,now.second));
16 }
17 }
18 return ans;
19 }

```

14 other language

14.1 java

14.1.1 文件操作

```

1 import java.io.*;
2 import java.util.*;
3 import java.math.*;
4 import java.text.*;
5
6 public class Main{
7
8     public static void main(String args[]){
9         throws FileNotFoundException,
10         IOException
11         Scanner sc = new Scanner(new FileReader(
12             "a.in"));
13         PrintWriter pw = new PrintWriter(new
14             FileWriter("a.out"));
15         int n,m;
16         n=sc.nextInt();//读入下一个INT
17         m=sc.nextInt();
18
19         for(ci=1; ci<=c; ++ci){
20             pw.println("Case #"+ci+": easy for
21                 output");
22         }
23
24         pw.close();//关闭流并释放，这个很重要，
25         否则是没有输出的
26         sc.close();//关闭流并释放
27     }
28 }

```

14.1.2 优先队列

```

1 PriorityQueue queue = new PriorityQueue( 1,
2     new Comparator(){
3         public int compare( Point a, Point b ){
4             if( a.x < b.x || a.x == b.x && a.y < b.y )
5                 return -1;
6             else if( a.x == b.x && a.y == b.y )
7                 return 0;
8             else return 1;
9         }
10    });

```

14.1.3 Map

```

1 Map map = new HashMap();
2 map.put("sa","dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj );
7 }

```

14.1.4 sort

```

1 static class cmp implements Comparator{
2     public int compare(Object o1,Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8 public static void main(String[] args)
9     throws IOException{
10     Scanner cin = new Scanner(System.in);
11     int n;
12     n=cin.nextInt();
13     BigInteger[] seg = new BigInteger[n];
14     for (int i=0;i<n;i++)
15         seg[i]=cin.nextBigInteger();
16     Arrays.sort(seg,new cmp());
17 }

```

14.2 python heap

```

1 import heapq
2
3 heap = [7,1,2,2]
4 heapq.heapify(heap)
5 print(heap) # [1, 2, 2, 7]
6 heapq.heappush(heap, 5)
7 print(heap) # [1, 2, 2, 7, 5]
8 print(heapq.heappop(heap)) # 1
9 print(heap) # [2, 2, 5, 7]

```

14.3 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

14.4 python output

```

1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB "{}!"'.format('Jin', 'Kela'
10     ''))
11 # AAA Jin BBB "Kela!"
12
13 hello = 'hello, world\n'
14 hellos = repr(hello)
15 print(hellos) # 'hello, world\n'
16
17 x = 32.5
18 y = 40000
19 print(repr((x, y, ('spam', 'eggs'))))
20 # "(32.5, 40000, ('spam', 'eggs'))"
21
22 x = 7
23 print(eval('3 * x')) # 21

```

15 zformula

15.1 formula

15.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

15.1.2 圖論

- 對於平面圖， $F = E - V + C + 1$ ， C 是連通分量數
- 對於平面圖， $E < 3V - 6$
- 對於連通圖 G ，最大獨立點集的大小設為 $I(G)$ ，最大匹配大小設為 $M(G)$ ，最小點覆蓋設為 $C_v(G)$ ，最小邊覆蓋設為 $C_e(G)$ 。對於任意連通圖：

$$(a) \quad I(G) + C_v(G) = |V|$$

$$(b) \quad M(G) + C_e(G) = |V|$$

- 對於連通二分圖：

$$(a) \quad I(G) = C_v(G)$$

$$(b) \quad M(G) = C_e(G)$$

- 最大權閉合圖：

$$(a) \quad C(u, v) = \infty, (u, v) \in E$$

$$(b) \quad C(S, v) = W_v, W_v > 0$$

$$(c) \quad C(v, T) = -W_v, W_v < 0$$

$$(d) \quad \text{ans} = \sum_{W_v > 0} W_v - \text{flow}(S, T)$$

- 最大密度子圖：

$$(a) \quad \text{求 } \max \left(\frac{W_e + W_v}{|V|} \right), e \in E', v \in V'$$

- $U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$
- $C(u, v) = W_{(u, v)}, (u, v) \in E$ 雙向邊
- $C(S, v) = U, v \in V$
- $D_u = \sum_{(u, v) \in E} W_{(u, v)}$
- $C(v, T) = U + 2g - D_v - 2W_v, v \in V$
- 二分搜 g ：
 $l = 0, r = U, \text{eps} = 1/n^2$
if $(U \times |V| - \text{flow}(S, T))/2 > 0$ $l = \text{mid}$
else $r = \text{mid}$
- $\text{ans} = \min_cut(S, T)$
- $|E| = 0$ 要特殊判斷

- 弦圖：

- 點數大於 3 的環都要有一條弦
- 完美消除序列從後往前依次給每個點染色，給每個點染上可以染的最小顏色
- 最大團大小 = 色數
- 最大獨立集：完美消除序列從前往後能選就選
- 最小團覆蓋：最大獨立集的點和他延伸的邊構成
- 區間圖是弦圖
- 區間圖的完美消除序列：將區間按造又端點由小到大排序
- 區間圖染色：用線段樹做

15.1.3 dinic 特殊圖複雜度

- 單位流： $O \left(\min \left(V^{3/2}, E^{1/2} \right) E \right)$
- 二分圖： $O \left(V^{1/2} E \right)$

15.1.4 0-1 分數規劃

$$x_i \in \{0, 1\} \cdot x_i \text{ 可能會有其他限制} \cdot \text{求 } \max \left(\frac{\sum B_i x_i}{\sum C_i x_i} \right)$$

- $D(i, g) = B_i - g \times C_i$
- $f(g) = \sum D(i, g) x_i$
- $f(g) = 0$ 時 g 為最佳解， $f(g) < 0$ 沒有意義
- 因為 $f(g)$ 單調可以二分搜 g
- 或用 Dinkelbach 通常比較快

```

1 binary_search(){
2     while(r-l>eps){
3         g=(l+r)/2;
4         for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i, g)
5         找出一組合法x[i]使f(g)最大;
6         if(f(g)>0) l=g;
7         else r=g;
8     }
9     Ans = r;
10 }
11 Dinkelbach(){
12     g=任意狀態(通常設為0);
13     do{
14         Ans=g;
15         for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i, g)
16         找出一組合法x[i]使f(g)最大;
17         p=0, q=0;
18         for(i:所有元素)

```

```

19 |         if(x[i])p+=B[i],q+=C[i];
20 |         g=p/q;//更新解·注意q=0的情況
21 |     }while(abs(Ans-g)>EPS);
22 |     return Ans;
23 | }

```

15.1.5 學長公式

- $\sum_{d|n} \phi(n) = n$
- $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
- Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
- $\gamma = 0.57721566490153286060651209008240243104215$
- 格雷碼 $n \oplus (n >> 1)$
- $SG(A+B) = SG(A) \oplus SG(B)$
- 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

15.1.6 基本數論

- $\sum_{d|n} \mu(n) = [n == 1]$
- $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
- $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
- $\sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) = n \sum_{d|n} d \times \phi(d)$

15.1.7 排組公式

- k 卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
- $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
- Stirling number of 2^{nd} , n 人分 k 組方法數目
 - $S(0, 0) = S(n, n) = 1$
 - $S(n, 0) = 0$
 - $S(n, k) = kS(n-1, k) + S(n-1, k-1)$
- Bell number, n 人分任意多組方法數目
 - $B_0 = 1$
 - $B_n = \sum_{i=0}^n S(n, i)$
 - $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
 - $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
 - $B_{p^m+n} \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
 - From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$
- Derangement, 錯排, 沒有人在自己位置上
 - $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
 - $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
 - From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$
- Binomial Equality
 - $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n}$

- $\sum_k \binom{l}{m+k} \binom{s}{n+k} = \binom{l+s}{l-m+n}$
 - $\sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
 - $\sum_{k \leq l} \binom{l-k}{m} \binom{s}{n} (-1)^k = \frac{(-1)^{l+m} \binom{s-m-1}{l-n-m}}{(-1)^{l+m} \binom{s-m-1}{l-n-m}}$
 - $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
 - $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
 - $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
 - $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n}$
 - $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
 - $\sum_{k \leq m} \binom{m+r}{k} x^k y^k = \sum_{k \leq m} \binom{r}{k} (-x)^k (x+y)^{m-k} =$
- 模擬賽遇到的
 - 對於一個長度為 L 的環·使用 k 種顏色進行相鄰不同色的著色·其著色方法數的公式為：

$$P(L, k) = (k-1)^L + (-1)^L (k-1)$$

15.1.8 幕次, 幕次和

- $a^{b \% p} = a^{b \% \varphi(p) + \varphi(p)}, b \geq \varphi(p)$
- $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
- $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
- $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
- $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
- $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
- $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
- 除了 $B_1 = -1/2$ ·剩下的奇數項都是 0
- $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

15.1.9 Burnside's lemma

- $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
- $X^g = t^{c(g)}$
- G 表示有幾種轉法· X^g 表示在那種轉法下·有幾種是會保持對稱的· t 是顏色數· $c(g)$ 是循環節不動的面數。
- 正立方體塗三顏色·轉 0 有 3^6 個元素不變·轉 90 有 6 種·每種有 3^3 不變·180 有 $3 \times 3^4 \cdot 120(\text{角})$ 有 $8 \times 3^2 \cdot 180(\text{邊})$ 有 $6 \times 3^3 \cdot \text{全部}$ $\frac{1}{24} (3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) =$

15.1.10 Count on a tree

- Rooted tree: $s_{n+1} = \frac{1}{n} \sum_{i=1}^n (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$
- Unrooted tree:
 - Odd: $a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$
 - Even: $Odd + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$
- Spanning Tree
 - 完全圖 $n^n - 2$
 - 一般圖 (Kirchhoff's theorem) $M[i][i] = \text{degree}(V_i), M[i][j] = -1, \text{if have } E(i, j), 0 \text{ if no edge. delete any one row and col in } A, \text{ans} = \det(A)$

16 Интернационал

16.1 保佑

```

1 |
2 |
3 |
4 |
5 |
6 |
7 |
8 |
9 |
10 |
11 |
12 |
13 |
14 |
15 | //
16 | //
17 | //
18 | //
19 | //
20 | //
21 | //
22 | //
23 | //
24 | //
25 | //
26 | //
27 | //
28 | //
29 | //
30 | //
31 | //
32 | //
33 | //
34 | //
35 | //
36 | //
37 | //
38 | //
39 | //
40 | //

```

佛祖保佑 永無BUG

```

41 |
42 |
43 |
44 |
45 |
46 |
47 |
48 |
49 |
50 |
51 |
52 |
53 |
54 |
55 | #
56 | #
57 | #
58 | #
59 | #
60 | #
61 | #
62 | #
63 | #
64 | #
65 | #
66 | #
67 | #
68 | #
69 | #
70 | #
71 | #
72 | #
73 | #
74 | #
75 |
76 |
77 | // ## #####
78 | // ## ##
79 | // ## ##
80 | // ## ##
81 | // ## ##
82 | // ## ##
83 | // ## ##
84 | // ## ##
85 | // #####
86 | // ## ##
87 | // ## ##
88 | // ## ##
89 | // ## ##
90 | // ## ##
91 | // ## ##
92 | // ## ##
93 | // #####
94 | //
95 | // 元首保佑 永無BUG
96 |
97 | //
98 | //
99 | //
100 | //
101 | //

```

神獸保佑 永無BUG!

```
102| //      u/u/u/****/u\u\u
103| //      u/u/****/\u\u
104| //      /*/*
105| //      | |
106| //      /+--+\\
107| //      || 卐 ||
108| //      \\==//
109| //
110| //      神獸保佑 永無BUG
```

ACM ICPC Team Reference - CCU_BABABOY

Contents

1 Computational Geometry	1	3 Flow	4	7 String	10	13 other	14
1.1 basic	1	3.1 dinic	4	7.1 ac 自動機	10	13.1 WhatDay	14
1.2 convex-hull	1	3.2 dinic flow decomposition	4	7.2 hash	10	13.2 上下最大正方形	14
1.3 Line Segment Intersection	1	3.3 Edmonds-Karp	5	7.3 KMP	10	13.3 最大矩形	14
1.4 Point in Polygon	1	3.4 Min Cost Max-Flow	5	7.4 manacher	10		
1.5 動態開點李超樹	1			7.5 suffix-array	10		
1.6 半平面交集	1	4 Graph	5	7.6 Zfunction	11	14 other language	15
1.7 最近點對	2	4.1 bellman-ford	5			14.1 java	15
1.8 李超線段樹	2	4.2 dijk	5	8 Tarjan	11	14.1.1 文件操作	15
1.9 矩形覆蓋面積	2	4.3 Euler-Path	5	8.1 dominator tree	11	14.1.2 優先隊列	15
		4.4 SCC	5	8.2 tnfsb017 2 sat	11	14.1.3 Map	15
		4.5 tarjan scc	6	8.3 橋連通分量	11	14.1.4 sort	15
		4.6 線段樹優化建圖	6	8.4 雙連通分量 & 割點	12	14.2 python heap	15
						14.3 python input	15
						14.4 python output	15
2 Data Structure	2	5 Linear Programming	6	9 Tree Problem	12		
2.1 DSU-rollback	2	5.1 simplex	6	9.1 centroid decomposition	12	15 zformula	15
2.2 FHQ-treap	2			9.2 find-centroid	12	15.1 formula	15
2.3 persistent-segment-tree	3	6 Number Theory	6	9.3 hld	13	15.1.1 Pick 公式	15
2.4 segment tree lazytag	3	6.1 basic	6	9.4 lca	13	15.1.2 圖論	15
2.5 trie	3	6.2 bit set	7			15.1.3 dinic 特殊圖複雜度	15
2.6 Weighted DSU	4	6.3 cantor expansion	7	10 compare	13	15.1.4 0-1 分數規劃	15
2.7 動態開點線段樹	4	6.4 FFT	7	10.1 check	13	15.1.5 學長公式	16
		6.5 FFT2	7	10.2 gen	13	15.1.6 基本數論	16
		6.6 find real root	8			15.1.7 排組公式	16
		6.7 FWT	8	11 default	13	15.1.8 冪次, 冪次和	16
		6.8 LinearCongruence	8	11.1 debug	13	15.1.9 Burnside's lemma	16
		6.9 Lucas	8	11.2 ext	14	15.1.10 Count on a tree	16
		6.10 Matrix	8	11.3 IncStack	14		
		6.11 MillerRobin	9	11.4 input	14	16 Интернационал	16
		6.12 NTT	9			16.1 保佑	16
		6.13 Simpson	9	12 dp 優化	14		
		6.14 外星模運算	9	12.1 2D1D 優化	14		
		6.15 數位統計	9	12.2 CHT	14		
		6.16 質因數分解	10	12.3 分治優化	14		

ACM

ICPC Judge Test - CCU_BABABOY

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6 const size_t KB = 1024;
7 const size_t MB = KB * 1024;
8 const size_t GB = MB * 1024;

```

```

9 size_t block_size, bound;
10 void stack_size_dfs(size_t depth = 1) {
11     if (depth >= bound)
12         return;
13     int8_t ptr[block_size]; // 若無法編譯將
14                             // block_size 改成常數
15     memset(ptr, 'a', block_size);
16     cout << depth << endl;
17     stack_size_dfs(depth + 1);
18 }
19
20 void stack_size_and_runtime_error(size_t
21     block_size, size_t bound = 1024) {
22     system_test::block_size = block_size;
23     system_test::bound = bound;
24     stack_size_dfs();
25 }
26
27 double speed(int iter_num) {
28     const int block_size = 1024;
29     volatile int A[block_size];
30     auto begin = chrono::high_resolution_clock
31         ::now();
32     while (iter_num--)
33         for (int j = 0; j < block_size; ++j)
34             A[j] += j;
35     auto end = chrono::high_resolution_clock::
36         now();

```

```

34     chrono::duration<double> diff = end -
35         begin;
36     return diff.count();
37 }
38
39 void runtime_error_1() {
40     // Segmentation fault
41     int *ptr = nullptr;
42     *(ptr + 7122) = 7122;
43 }
44
45 void runtime_error_2() {
46     // Segmentation fault
47     int *ptr = (int *)memset;
48     *ptr = 7122;
49 }
50
51 void runtime_error_3() {
52     // munmap_chunk(): invalid pointer
53     int *ptr = (int *)memset;
54     delete ptr;
55 }
56
57 void runtime_error_4() {
58     // free(): invalid pointer
59     int *ptr = new int[7122];
60     ptr += 1;
61     delete[] ptr;

```

```

62
63 void runtime_error_5() {
64     // maybe illegal instruction
65     int a = 7122, b = 0;
66     cout << (a / b) << endl;
67 }
68
69 void runtime_error_6() {
70     // floating point exception
71     volatile int a = 7122, b = 0;
72     cout << (a / b) << endl;
73 }
74
75 void runtime_error_7() {
76     // call to abort.
77     assert(false);
78 }
79
80 } // namespace system_test
81
82 #include <sys/resource.h>
83 void print_stack_limit() { // only work in
84     Linux
85     struct rlimit l;
86     getrlimit(RLIMIT_STACK, &l);
87     cout << "stack_size = " << l.rlim_cur << "
88         byte" << endl;

```